

Observational Production Functions for Language Models and the Measurement of Algorithmic Progress

By YIGIT OKAR*

Estimates of returns to compute and algorithmic progress in language models come mostly from models that developers chose to train. We ask what such data identify. Matched to the design of released models, cross-lab returns to compute do not differ detectably from an experimental benchmark in a low-power test; the split between parameters and data is not reliably estimated. In the leading data on algorithmic progress, the rate is weakly identified: bias and rate of technical change trade off along a ridge. Rebalancing away from under-training raised the training-cost efficiency of large models 1.1 to 4.9-fold, depending on the technology.

JEL: C51, D24, L86, O33, O47

Keywords: production functions, transmission bias, algorithmic progress, productivity dispersion, biased technical change, large language models

Much of what economists and forecasters know about how language models improve with scale, and about how fast the methods for training them improve, comes from models that their developers chose to train. Throughout, we call the firms and research groups that train language models “labs”; the term covers both the developers that release models and the designers of the training experiments that measure the technology. Observational scaling studies regress the benchmark scores of models released by many labs on their parameters and training tokens (Ruan, Maddison and Hashimoto, 2024; Maia Polo et al., 2025; Owen, 2024; Mertens, Fischl-Lanzoni and Thompson, 2026). The leading estimate of algorithmic progress regresses the perplexity of published models on their inputs and publication dates (Ho et al., 2024). To an economist these are production-

* Care AI (email: okar.yigit@gmail.com). This paper is a companion to Okar (2026), which estimates the technology of training language models from designed experiments; its material appeared as a section of an earlier version of that paper. This paper was produced in close collaboration with Claude, an AI system developed by Anthropic. Claude designed and carried out the analysis (the data construction, the theory and its verification, the estimation, the simulations and the replication of Ho et al. (2024)) and drafted the paper with the author, who takes responsibility for its content. The author posed the research question and made the project’s decisions on scope, disclosure and publication. The author gratefully acknowledges this contribution; under the AEA’s authorship policy an AI system cannot be listed as an author. The author thanks the authors of Ho et al. (2024) for making their data and code public, which made the exact replication in Section VI possible, and will share the findings of that section with them before circulating this paper and report their response in a revised version. Data and code availability: all data used in the paper are publicly available. The replication package (github.com/yigitokar/scaling-laws-as-production-functions, release v4) reproduces the tables and figures from the raw downloads with fixed random seeds. For sources that carry no license permitting redistribution (Epoch AI’s digitization of the Chinchilla runs and the Farseer run files), it provides download scripts and SHA-256 checksums of the files we used in place of the data.

function regressions on observational data. The labs chose their inputs knowing something about their own productivity. That is the setting in which [Marschak and Andrews \(1944\)](#) showed least squares to be biased (see [Griliches and Mairesse, 1998](#)), and [Whitfill \(2025\)](#) and [König et al. \(2026\)](#) warn that the estimates for language models may be confounded.

This paper asks what such data can identify. The industry offers a check that economists who study production rarely have. Besides observational data there are designed experiments: public sweeps in which researchers train models over grids of parameters and tokens with a fixed recipe ([Hoffmann et al., 2022](#); [Bhagia et al., 2025](#); [Gadre et al., 2025](#)). A companion paper ([Okar, 2026](#)) uses such experiments to estimate the training technology and reads labs’ choices through it. Here we use them as a benchmark for observational estimators, and we derive what cross-lab data can and cannot deliver when labs choose inputs knowing their productivity.

We have five findings. First, returns to compute survive a comparison with the experiments once the design is matched. The technology is strongly curved, and released models sit where an extra training token is worth little, so a comparison of observational estimates with global experimental elasticities mistakes curvature for bias. We therefore evaluate the experimental technology at the inputs of the released models and apply each estimator to both sets of outputs. On 57 models the compute elasticity of HellaSwag performance differs by -0.001 , with a standard error of 0.041 . This is a failure to reject in a joint test of low power, not evidence of unbiasedness: the bias detectable with 80 percent power is 37 percent of the benchmark. The split of returns between parameters and data is not reliably estimated.

Second, compute is measured well enough that errors in variables do not matter, but the standard remedies for transmission bias cannot be applied. The available instruments are weak or fail their exclusion restrictions, and product lines are too short for dynamic-panel estimators. Proxy inversion fails for structural reasons that we establish: at given compute, Hicks-neutral productivity does not move the input mix, and compute cannot proxy productivity because the inputs on the expansion path are functions of compute alone.

Third, productivity dispersion across model families is about as large as within US manufacturing industries when it is measured in output units. Measured in compute-equivalent units, the 90–10 ratio is 7 to 24, against 1.9 in manufacturing. Because returns to compute are small, the two unit systems differ by a large factor, and comparisons with other industries are a choice of cardinalization rather than a finding.

Fourth, in the data of [Ho et al. \(2024\)](#) the rate of algorithmic progress is weakly identified. The sample is off the expansion path, and the estimates lie on a ridge along which the bias of technical change trades off against its rate, the identification problem of [Diamond, McFadden and Rodriguez \(1978\)](#). The profile-likelihood interval for the doubling time of effective compute runs from 4.1 to 40.5

months, and a restriction on the bias buys most of the precision: imposing Hicks neutrality gives 9 to 12 months. We reproduce the published estimates exactly from the authors’ data and code. On this ridge the reported point depends on the optimizer’s stopping rule: run to convergence, the authors’ objective falls from 0.0518 to 0.0507 while the fit changes by 0.27 percent, and the doubling time moves from 8.7 to 6.1 months. The finding concerns numerical optimization on a flat objective, not the order of magnitude of progress.

Fifth, counting only the elimination of under-training, the move from the allocations implied by the scaling law of [Kaplan et al. \(2020\)](#) to those of [Hoffmann et al. \(2022\)](#) raised the training-cost efficiency of large models by a factor of 2.0 under the reference technology of the companion paper (1.1 to 4.9 across 14 technologies). [Gundlach et al. \(2025\)](#) define the compute-equivalent gain of such rebalancing and show that continuing the earlier rule would cost about tenfold at 2025 frontier compute. Our counterfactual agrees; what we add is that the gain realized between the two eras was a fraction of that counterfactual.

Section I collects the theory that organizes these findings. Transmission from productivity to compute biases returns to compute with the sign of the compute response, upward when more productive labs raise more compute and downward when labs train to a common capability target, but it leaves the allocation exponent unbiased. The forward–reverse bracket familiar from errors in variables ([Klepper and Leamer, 1984](#)) fails under funding, and selection on released outcomes attenuates transmission bias and can reverse it. If compute-optimal models were observed at several dates, the drift in their ratio of tokens to parameters would identify the sign of the bias of technical change, though not its size, because the elasticity of substitution between parameters and data is below one at interior optima. Growth-accounting corollaries characterize when compute-equivalent gains are the same at every scale, how regressions of frontier loss on compute without time effects attribute progress to scale ([Nagy et al., 2013](#)), and how an equal-weight productivity residual misattributes input shifts to productivity ([Hall, 1988](#)).

These are classic questions of production economics. Estimators of production functions under endogenous inputs ([Olley and Pakes, 1996](#); [Levinsohn and Petrin, 2003](#); [Akerberg, Caves and Frazer, 2015](#); [Gandhi, Navarro and Rivers, 2020](#)) are usually judged by their assumptions, because the truth is unknown. Here part of the truth is measured, so the exercise is a validation of observational estimators against experimental benchmarks, in the tradition of [LaLonde \(1986\)](#) and [Dehejia and Wahba \(1999\)](#), but across populations ([Hotz, Imbens and Mortimer, 2005](#); [Todd and Wolpin, 2006](#)): the experimental recipes are not the released models’ recipes. The measurement of productivity dispersion ([Syverson, 2004, 2011](#)) and of the bias of technical change ([Diamond, McFadden and Rodriguez, 1978](#); [León-Ledesma, McAdam and Willman, 2010](#)) meet a technology whose returns to its composite input are far from constant. For the literature on progress in machine learning ([Erdil and Besiroglu, 2022](#); [Sevilla et al., 2022](#); [Ho et al., 2024](#);

(Gundlach et al., 2025; Whitfill, 2025), the paper supplies the identification results and the benchmark. The general lesson reaches beyond language models. Whenever producers choose inputs knowing their productivity and designed variation exists, as in agronomic field trials or sweeps of chip designs, matching the observational design to the experimental one is what makes a validation informative. Where designed variation does not exist, what observational data lack is variation transverse to the expansion path, and no estimator supplies it.

The rest of the paper proceeds as follows. Section I sets out the theory, Section II the data. Section III compares observational and experimental returns to compute, Section IV discusses the measurement of compute and the standard remedies, and Section V measures productivity dispersion. Section VI revisits the estimation of algorithmic progress, Section VII separates allocative from technical change, and Section VIII concludes. The appendices collect the proofs (Appendix A), details of the data (Appendix B), the simulations (Appendix C) and additional tables (Appendix D).

I. What Cross-Lab Data Can Identify

A. The Technology

We use the technology of Hoffmann et al. (2022) with productivity terms, in the generalized form of Okar (2026). A training run uses N parameters and D training tokens (tokens processed, including repetitions) and attains held-out cross-entropy

$$(1) \quad L = E + e^{-\omega} \left[A(e^{\psi_N} N)^{-\alpha} + B(e^{\psi_D} D)^{-\beta} \right]^{\kappa} e^{\zeta}, \quad A, B, \alpha, \beta, \kappa > 0,$$

where E is irreducible loss, ω is Hicks-neutral productivity, ψ_N and ψ_D are factor-augmenting productivity, and ζ is noise. The Chinchilla form is $\kappa = 1$. The lab knows (ω, ψ_N, ψ_D) when it chooses inputs, but not ζ . Lower-case letters denote logs: $n = \ln N$, $d = \ln D$ and $c = \ln C$, with training compute $C = 6ND$, so that $c = \ln 6 + n + d$; $M \equiv D/N$ is the number of tokens per parameter. The output index is $y \equiv -\ln(L - E) = \omega + F(n + \psi_N, d + \psi_D) - \zeta$, with $F(n, d) \equiv -\kappa \ln(Ae^{-\alpha n} + Be^{-\beta d})$.

The output elasticities are $\varepsilon_N \equiv \partial y / \partial n$ and $\varepsilon_D \equiv \partial y / \partial d$, and their ratio is the wedge $w \equiv \varepsilon_N / \varepsilon_D$. A run on the training-cost-minimizing expansion path has $w = 1$. Along that path parameters grow with compute at elasticity $a \equiv \beta / (\alpha + \beta)$, reducible loss falls with compute at elasticity $\gamma \equiv \kappa \alpha \beta / (\alpha + \beta)$, which we call the frontier elasticity, and the elasticity of substitution between parameters and data is $\sigma^* \equiv 2 / (2 + \alpha + \beta)$. Productivity enters the allocation through the bias index $\chi \equiv \beta \psi_D - \alpha \psi_N$ and output through compute-equivalent productivity $\Omega \equiv \omega + \gamma(\psi_N + \psi_D)$.

Lemma 1 (Allocation and allocative loss). *(i) At given compute, the inputs that maximize y satisfy $n^* = \ln G + a(c - \ln 6) + \chi / (\alpha + \beta)$ and $d^* = c - \ln 6 -$*

n^* , where $G \equiv (\alpha A / \beta B)^{1/(\alpha+\beta)}$, and attain $y^* = \Omega + \gamma(c - \ln 6) - \ln K - \zeta$ for a constant K . Hicks-neutral productivity does not enter the allocation.

(ii) For any run, $\ln w = \frac{\alpha+\beta}{2} \ln(M/M^*(C))$, where $M^*(C)$ is the compute-optimal ratio at the run's compute for the lab's own productivity.

(iii) Let $C_{\min}(R)$ be the least compute that attains the run's reducible loss $R = L - E$. Then

$$(2) \quad \frac{C}{C_{\min}(R)} = \left(\frac{\alpha + \beta w}{\alpha + \beta} \right)^{(\alpha+\beta)/(\alpha\beta)} w^{-1/\alpha},$$

which depends on neither (A, B, E, κ) nor productivity. To second order, $\ln(C/C_{\min}) = (\ln w)^2 / [2(\alpha + \beta)]$.

Part (iii) makes allocative efficiency measurable: the compute a run spends beyond the cheapest way of reaching its own loss is a function of its wedge alone. The loss is a Harberger triangle whose area is governed by the elasticity of substitution, since $2(\alpha + \beta) = 4(1 - \sigma^*)/\sigma^*$.

Why would a wedge differ from one? Okar (2026, Proposition 1) shows that a lab that values compactness (because it serves the model, because its users bear the serving cost and adoption responds to it, or because it targets a deployment tier) chooses $w > 1$, and that serving demand cannot produce $w < 1$ as long as serving expenditure rises with model size at given quality. Wedges below one arise from optimization against a different technology, for example the law of Kaplan et al. (2020), from a binding cap on training data, or from costs of data. For an econometrician who ignores factor-augmenting productivity, the measured wedge is $we^{-\chi}$. Most open-weight models released since 2023 are trained beyond training-cost minimization (Okar, 2026), so released models are far from the expansion path.

B. Transmission and Selection

Consider first runs on the expansion path of a common technology with Hicks-neutral productivity only. By Lemma 1(i), $y_i = \omega_i + \gamma c_i - \zeta_i$ up to a constant. Compute responds to productivity through $c_i = \pi_0 + \pi_1 \omega_i + \eta_i$, with (ω, η, ζ) mutually uncorrelated and variances $V_\omega, V_\eta, V_\zeta$. Three rules are special cases: exogenous budgets ($\pi_1 = 0$); funding, under which more productive labs raise more compute ($\pi_1 > 0$); and a capability target, under which every lab trains until it reaches a common loss ($\pi_1 = -1/\gamma$).

Proposition 1 (Transmission). (i) The least-squares slope of y on c converges to $b_f = \gamma + \pi_1 V_\omega / (\pi_1^2 V_\omega + V_\eta)$; its bias has the sign of π_1 . Under a capability target, $b_f = \gamma V_\eta / (V_\omega / \gamma^2 + V_\eta) \in [0, \gamma)$, and if every lab hits the target exactly ($V_\eta = 0$), cross-lab data carry no information about γ .

(ii) The residual variance is $V_\omega V_\eta / \text{Var}(c) + V_\zeta$, so productivity dispersion measured from residuals is understated whenever $\pi_1 \neq 0$.

(iii) The reverse-regression estimate $\gamma_r \equiv \text{Var}(y) / \text{Cov}(c, y)$ satisfies

$$(\gamma_r - \gamma) \text{Cov}(c, y) = (1 + \gamma\pi_1)V_\omega + V_\zeta.$$

When $\text{Cov}(c, y) > 0$, the interval $[b_f, \gamma_r]$ contains γ if and only if $-(1 + V_\zeta/V_\omega)/\gamma \leq \pi_1 \leq 0$; under funding both estimates exceed γ .

(iv) The least-squares slope of n on c equals a for every π_1 . With factor-biased productivity it converges to $a + \text{Cov}(\chi, c) / [(\alpha + \beta) \text{Var}(c)]$.

Transmission biases returns to compute, not the allocation. Upward bias requires that more productive labs command more compute; downward bias requires that labs stop when a target is reached. In the second case the data can be entirely uninformative, because compute then varies only to offset productivity. Part (iii) matters in practice: the forward and reverse regressions bracket the truth under errors in variables (Klepper and Leamer, 1984), and they continue to do so under a capability target, but not under funding.

Proposition 1 places runs on the expansion path. Released models are far off it, and their log wedges enter output through the allocative loss of Lemma 1(iii). Two consequences follow by linear projection. When the regression is run on both inputs, the allocative loss is part of the technology F , and transmission loads on the input mix only if labs' wedges covary with their productivity at given compute. If they do not, the projection of ω on (n, d) runs along $c = \ln 6 + n + d$, and transmission shifts the two input elasticities by the same amount. If more productive labs also over-train more (or less), the two biases differ and can have opposite signs. When the regression is run on compute alone, it imposes equal weights on the two inputs, and the productivity residual inherits the Hall-type term of Corollary 1(iii) below.

Labs also release selectively. Suppose a run is released only if $y_i \geq \bar{y}$, and let $S_i \equiv \omega_i - \zeta_i$.

Proposition 2 (Selection on the outcome). (i) With exogenous budgets ($\pi_1 = 0$), the least-squares slope in released samples is at most γ ; if S has a log-concave density, it lies in $[0, \gamma]$.

(ii) Let (ω, η, ζ) be jointly normal, $\delta \equiv \text{Cov}(c, y) / \text{Var}(y)$, $V_{c|y} \equiv \text{Var}(c) - \delta^2 \text{Var}(y)$ and $r \equiv \text{Var}(y \mid y \geq \bar{y}) / \text{Var}(y) \in (0, 1)$. Selection leaves the reverse slope δ unchanged and moves the forward slope to

$$b_{\text{sel}} = \frac{\delta r \text{Var}(y)}{\delta^2 r \text{Var}(y) + V_{c|y}},$$

which increases with r . If $\pi_1 \leq 0$ and $\text{Cov}(c, y) > 0$, then $b_{\text{sel}} \leq b_f \leq \gamma$. If

$\pi_1 > 0$, then $b_{\text{sel}} > \gamma$ if and only if

$$r > r^* \equiv \frac{\gamma V_{c|y}}{\delta \text{Var}(y)(1 - \gamma\delta)}.$$

(iii) In the Gaussian case, the bracket of Proposition 1(iii) remains valid in released samples whenever $-(1 + V_\zeta/V_\omega)/\gamma \leq \pi_1 \leq 0$.

Part (i) is the classical attenuation of least squares under truncation on the dependent variable (Hausman and Wise, 1977; Goldberger, 1981). What part (ii) adds is the interaction with transmission. Under funding, mild selection leaves the upward transmission bias dominant and severe selection reverses it; under exogenous budgets or a target, the net bias is negative, and the forward–reverse bracket survives in released samples.

C. Why Proxy Estimators Are Unavailable

Proxy estimators (Olley and Pakes, 1996; Levinsohn and Petrin, 2003; Akerberg, Caves and Frazer, 2015) recover productivity by inverting a policy function. They need a state variable, a flexible input and a timing assumption. For language models the natural state variable is the lab’s compute capacity, the provisioned cluster, which is fixed before a run. The flexible input is the run’s split of its compute budget between parameters and tokens, chosen at the start of the run given capacity and the recipe’s quality. Release follows the observed outcome. Suppose that runs minimize training cost with a common wedge $\bar{w}$, share $(A, B, \alpha, \beta, \kappa)$, and differ in $(\omega_i, \psi_{N,i}, \psi_{D,i})$.

Proposition 3 (Proxies). (i) $\ln M_i = m_0(c_i) + \frac{2}{\alpha+\beta}(\ln \bar{w} - \chi_i)$, with $m_0(c) = -2 \ln G + (1 - 2a)(c - \ln 6)$. At given compute the input mix reveals the bias index χ_i exactly but does not depend on ω_i : no allocation-based proxy exists for Hicks-neutral productivity.

(ii) The map $(n, d) \mapsto (c, \ln M)$ is a bijection, so a first-stage control function in $(c, \ln M)$ absorbs $F(n, d)$ entirely: the functional-dependence problem of Akerberg, Caves and Frazer (2015). The same holds for compute as a proxy for ω under a strictly monotone budget rule $c = h(\omega, z)$, because on the expansion path the inputs are functions of c alone.

Identification would therefore have to come from timing, for example compute committed before a recipe innovation, or from observed investment in capacity, and public data record neither. Under a capability target the mix chosen for a given loss does co-move with ω when $\alpha \neq \beta$, because more productive labs need less compute and the compute-optimal mix changes with compute. That co-movement runs entirely through compute: conditional on c , the mix is flat in ω .

D. Technical Change

Diamond, McFadden and Rodriguez (1978) show that time series of outputs, inputs and marginal products cannot separate the elasticity of substitution from the bias of technical change. Compute-optimal models escape half of that problem.

Proposition 4 (Technical change). *(i) Let output $y = F(n, d, t)$ be smooth, with $F_n, F_d > 0$ and an interior compute-optimal allocation at each date. Let $\mathcal{B}_t \equiv \partial \ln(F_n/F_d)/\partial t$ at fixed (n, d) be the Hicks bias of technical change, which is parameter-using if $\mathcal{B}_t > 0$. Then $\sigma_t^* \in (0, 1)$ and*

$$\frac{\partial \ln M^*(C, t)}{\partial t} = -\frac{\sigma_t^*}{1 - \sigma_t^*} \mathcal{B}_t.$$

The sign of the bias is identified from compute-optimal models alone: $\mathcal{B}_t > 0$ if and only if the compute-optimal number of tokens per parameter falls at given compute. Its size is not identified without σ_t^ .*

(ii) In the family (1) with $\psi_{N,t} = g_N t$ and $\psi_{D,t} = g_D t$, the frontier shifts at rate $\gamma(g_N + g_D)$, the mix drifts at $\partial \ln M^/\partial t = 2[(1 - \alpha)g_N - \alpha g_D]$, and $\mathcal{B} = \beta g_D - \alpha g_N$. The drift of M^* therefore signs $\beta g_D - \alpha g_N$, which is the sign of $g_D - g_N$ only when $\alpha = \beta$. Given α and γ at each date, g_N and g_D are identified separately, by the family's functional form; $\alpha + \beta$, and with it σ^* and $|\mathcal{B}|$, is not.*

(iii) Hicks-neutral technical change ($\alpha g_N = \beta g_D$) holds if and only if $M^(C)$ is constant over time.*

The sign result works because the FLOP accounting fixes the relative cost of a parameter and a token, and because $\sigma^* < 1$ at interior optima. Part (ii) warns against a tempting shortcut: with $(\alpha, \beta) = (0.30, 0.45)$ and $(g_N, g_D) = (0.5, 0.4)$, $\mathcal{B} = 0.03 > 0$ and the compute-optimal D/N falls although parameters are augmented faster. The result is methodological. We do not take it to published compute-optimal laws of different vintages, which differ in tokenizers, parameter-counting conventions and outputs. It also requires that the effective relative cost of tokens not drift through data costs, data constraints or serving demand, so it applies to training-optimal allocations (IsoFLOP minima), not to released models. A designed IsoFLOP sweep at each vintage on a common output would identify the sign, and through the model-free curvature estimator of Okar (2026), the size.

E. Allocative versus Technical Change

A model's excess compute over best practice splits into an allocative and a technical part. Let run i have compute-equivalent productivity Ω_i and wedge w_i , and let $C_{\min, F}(L)$ be the compute that best practice (Ω_F) needs to reach loss L .

Proposition 5 (Farrell decomposition).

$$\ln(6N_i D_i) - \ln C_{\min, F}(L_i) = \underbrace{\Phi(w_i)}_{\text{allocative}} + \underbrace{(\Omega_F - \Omega_i)/\gamma + \zeta_i/\gamma}_{\text{technical}},$$

with $\Phi(w) = \ln(C/C_{\min})$ given by Lemma 1(iii). A reallocation at fixed productivity is purely allocative.

The move from the allocations of Kaplan et al. (2020) to those of Hoffmann et al. (2022) changed the mix, not the technology. Under the parameters of Besiroglu et al. (2024), Gopher’s allocation (280 billion parameters, 300 billion tokens) has $w = 0.36$ and used about twice the compute that its loss required ($\Phi = \ln 2.01$). Three corollaries concern growth accounting.

Corollary 1 (Growth accounting). (i) (Compute-equivalent gains.) Compare two technologies on their frontiers $L_j^*(C) = E_j + K_j(C/6)^{-\gamma_j}$. The gain $f(C) \equiv C_{\text{old}}^*(L_{\text{new}}^*(C))/C$ is the same at every compute level if and only if $E_{\text{old}} = E_{\text{new}}$ and $\gamma_{\text{old}} = \gamma_{\text{new}}$.

(ii) (Frontier regressions.) Let frontier releases satisfy $\ln R_t = \ln K - \gamma(c_t + g_A t)$, where g_A is algorithmic progress in compute-equivalent units, and $c_t = c_0 + gt + u_t$ with u_t independent of t . The least-squares slope of $-\ln R_t$ on c_t converges to $\gamma[1 + (g_A/g)R_{c,t}^2]$, where $R_{c,t}^2$ is the share of the variance of compute explained by time. With a deterministic trend it is $\gamma/(1 - s_A)$, where $s_A \equiv g_A/(g + g_A)$ is the algorithmic share of effective-compute growth.

(iii) (Hall-type bias.) To first order $\Delta y = \varepsilon_N \Delta n + \varepsilon_D \Delta d + \Delta \Omega$. A residual computed with equal weights $\bar{\varepsilon} = (\varepsilon_N + \varepsilon_D)/2$, as $C = 6ND$ suggests, equals $\Delta \Omega + \frac{1}{2}(\varepsilon_N - \varepsilon_D)(\Delta n - \Delta d)$. When over-trained models ($w > 1$) shift further toward tokens, the residual understates progress.

Part (i) says when a single number can summarize the gain of a new technology: the individual exponents may differ, but the irreducible loss and the frontier elasticity may not. Part (ii) is the scale-attribution bias of Nagy et al. (2013) in the frontier regressions of machine learning; adding a time trend recovers γ from deviations of compute from trend, and does so only if such deviations exist. Part (iii) is the analog of Hall (1988) for an industry whose accounting identity imposes equal cost weights on two inputs whose output elasticities differ.

II. Data

Released models. The observational panel has 128 dense base models from 38 families and 22 developers, released between March 2021 and July 2024. Benchmark scores come from the Open LLM Leaderboard, through the harmonized collections of Ruan, Maddison and Hashimoto (2024) and Maia Polo et al. (2025); all models share one evaluation harness and one set of prompts. Parameter counts are

the floating-point tensor counts of the Hugging Face checkpoints where available (86 models), excluding stored attention-mask buffers, and reported sizes otherwise. D is pretraining tokens processed, including repeated epochs and the tokens of a parent model for continued pretraining. Release dates come from Epoch AI (Epoch AI, 2026) for 90 models and from the Hugging Face upload date for 26; for the other 12, whose checkpoints were uploaded after release, we set them by hand. Mixture-of-experts models are excluded. The replication package lists every correction to the source data with its source. We flag distilled models (Gemma-2 2B and 9B), models trained on synthetic data (Phi and SmolLM) and code models. The identifying variation within families is thin: the within-family standard deviation of $\ln D$ is 0.42, against 1.29 for $\ln N$.

The models share no validation loss, so output is the log-odds of above-chance accuracy, $q = \ln[\tilde{a}/(1 - \tilde{a})]$, where $\tilde{a}$ is accuracy net of chance, rescaled to the unit interval and clipped to $[0.01, 0.99]$. The main benchmark is HellaSwag (Zellers et al., 2019), evaluated 10-shot. ARC-Challenge (25-shot) and Winogrande (5-shot) are secondary.

Designed runs. The experimental benchmark uses the final checkpoints of the 30 runs of the OLMo ladder (Bhagia et al., 2025), from 190 million to 3 billion parameters and from 0.5 to 10 times the Chinchilla ratio of tokens to parameters, and the 56 runs of Gadre et al. (2025) with at least 0.1 billion parameters, trained on three corpora. The OLMo runs are evaluated in the OLMES harness and the Gadre runs in LLM-foundry. The released OLMo-2 7B and 13B models (Team OLMo et al., 2024) come from the same pipeline as the ladder but were chosen by their developer; we use them only in an alternative benchmark. The 25 pretraining recipes of DataDecide (Magnusson et al., 2025) supply real outcomes for the semi-synthetic exercise of Section III.F.

Large models. For the allocative analysis we use the Epoch AI database of language models released from 2018 to September 2026, keeping 247 dense models with internally consistent parameters, compute and dataset size and at most four epochs, and setting $D = C/6N$ so that $6ND = C$ exactly.

Algorithmic progress. The data of Ho et al. (2024) have 231 model–benchmark observations from 144 papers on WikiText-103, Penn Treebank and WikiText-2, built with the authors’ own filters, which we re-implement from their public code.

III. Returns to Compute: A Design-Matched Validation

A. Specification

We estimate $q_i = \theta_0 + \theta_N n_i + \theta_D d_i + \omega_i + \nu_i$, which is Cobb–Douglas in the odds with Hicks-neutral productivity ω_i , and the regression of q_i on c_i alone. The slope θ_C of the latter is what observational scaling studies report. The estimators range from pooled least squares through year, developer, family and developer-by-period fixed effects to a notability-propensity control, a quadratic in the estimated propensity of Epoch notability, which is selection on observables rather than a

proxy correction.

B. Why the Design Must Be Matched

A comparison with global experimental elasticities misleads, because the technology is curved and released models sit in a different part of it. In the designed sweeps, HellaSwag log-odds are strongly concave in (n, d) : the F -statistic for the quadratic terms is 70 in the sweep of [Gadre et al. \(2025\)](#). On the experimental surface described below, at one billion parameters the local data elasticity falls from 0.57 at 20 tokens per parameter to 0.07 at 500. Released models are heavily over-trained, with a median of 213 tokens per parameter and a ninetieth percentile of 1,715, so they sit where an extra token is worth little. Global log-linear fits give observational data elasticities of 0.19 to 0.26, against 0.44 in the OLMo ladder and 0.51 in [Gadre et al. \(2025\)](#), but much of that gap is the technology, not bias.

We therefore match the design. We fit a quadratic surface $\hat{f}(n, d)$ with design intercepts to the 86 designed runs ($R^2 = 0.994$) and keep the released models inside the convex hull of the experimental inputs, the common-support trimming of [Dehejia and Wahba \(1999\)](#). We then apply each estimator twice, with the same fixed effects and controls: to the observed outputs, and to the outputs $q_i^* = \hat{f}(n_i, d_i)$ that the experimental technology produces with the same inputs. The difference is the estimator’s bias. The benchmark comes from other labs’ recipes, not from the treated population, so the exercise is a validation across populations in the sense of [Todd and Wolpin \(2006\)](#) and [Hotz, Imbens and Mortimer \(2005\)](#). The null hypothesis is joint: a common technology up to Hicks-neutral shifts, outputs that map one to one across evaluation harnesses, and no transmission or selection bias. Recipes differ in factor-biased ways ([Okar, 2026](#)), which would violate the first part, so a null bias can also reflect offsetting failures.

C. Support and Inference

The strictly experimental support is small ([Figure 1](#)). Without the released OLMo-2 models, the largest designed D is 921 billion tokens (one run of [Gadre et al., 2025](#) at 1.4 billion parameters), and above 3.8 billion parameters it is 634 billion. The strict support contains 20 released models from 9 families and 6 developers, all released between March 2021 and September 2023 with at most 6.7 billion parameters and 627 billion tokens; five of the nine families have two or more models, 16 models in all. The benchmark that includes OLMo-2 (88 runs) covers 57 models from 30 families and 19 developers, with at most 8.8 billion parameters. Fifteen of its families have two or more models (42 models) and fifteen are singletons. We report both designs.

Inference must allow for few clusters and for the uncertainty of the benchmark itself. Standard errors are CV3 jackknife standard errors by developer ([MacKinnon, Nielsen and Webb, 2023](#)). The benchmark’s uncertainty comes from a

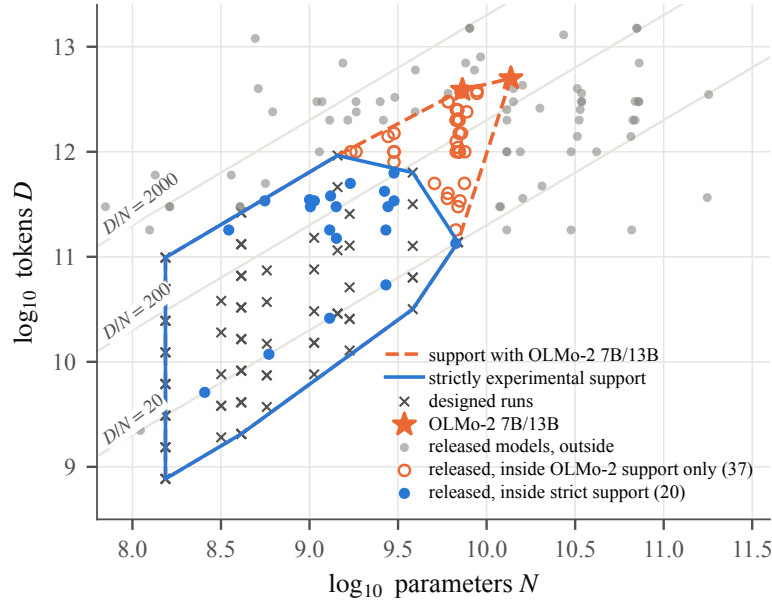


FIGURE 1. DESIGNED RUNS, RELEASED MODELS AND THE COMMON SUPPORT

Note: Log parameters and log training tokens of the designed runs (crosses: the OLMo ladder and [Gadre et al., 2025](#) with $N \geq 0.1B$) and of the 128 released base models (dots). Solid line: convex hull of the designed runs (the strictly experimental support; 20 released models inside, filled). Dashed line: hull including the released OLMo-2 7B and 13B models (stars; 37 more released models inside, open circles). Gray lines: 20, 200 and 2,000 tokens per parameter.

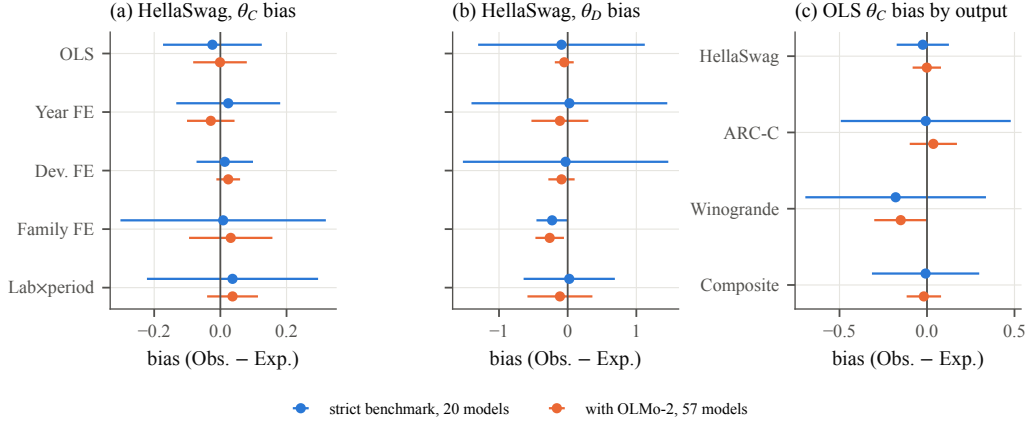


FIGURE 2. OBSERVATIONAL ESTIMATES AGAINST THE EXPERIMENTAL TECHNOLOGY ON THE SAME DESIGN

Note: Bias of each observational estimator relative to the same estimator applied to the outputs of the experimental technology on the same models (Obs. – Exp.), with 95 percent intervals (± 1.96 CV3 standard errors of the bias, including the benchmark’s uncertainty). Blue: strict benchmark (OLMo ladder and [Gadre et al., 2025](#); 20 models). Orange: benchmark including the released OLMo-2 7B and 13B models (57 models). Panels (a) and (b): HellaSwag, compute-only elasticity θ_C and data elasticity θ_D . Panel (c): pooled OLS bias in θ_C by output; ARC-Challenge and Winogrande use the censored (Tobit) benchmark. Estimates and conservative few-cluster p -values are in Table 1.

design-conditional wild bootstrap of the surface, clustered by design and model size. For each bias we compute three few-cluster p -values that include the benchmark’s uncertainty: CV3 with $t(G - 1)$ critical values, and restricted wild-cluster bootstrap- t tests studentized by CR1 or by CV3 ([Cameron, Gelbach and Miller, 2008](#); [Webb, 2023](#)). Table 1 reports the largest of the three, so a rejection requires all three to reject. The effective number of clusters ([Carter, Schnepel and Steigerwald, 2017](#)) is 2.5 to 6.7 of 19 on the 57-model design. On the strict design it is 1.2 to 3.8 of 6; there the three procedures disagree by up to a factor of ten, and none is reliable.

D. Returns to Compute

Once the design is matched, cross-lab returns to compute on HellaSwag do not differ detectably from the experimental technology (Table 1, Figure 2). On the 57-model design, pooled least squares gives $\theta_C = 0.328$, against 0.329 from the experimental technology on the same models, a bias of -0.001 (standard error 0.041) with $p = 0.98$. Across six estimators, from pooled least squares to developer-by-period fixed effects and the notability-propensity control, the biases lie between -0.029 and $+0.037$, and every p -value is at least 0.36. The strict surface evaluated on the same 57 models, an extrapolation for 37 of them, gives $+0.014$ (0.045), so the result does not rest on anchoring the surface with OLMo-2. On the strict support itself the point estimates do not move (from -0.024

TABLE 1. DESIGN-MATCHED COMPARISON OF OBSERVATIONAL AND EXPERIMENTAL ELASTICITIES

	Compute only, θ_C			θ_N	θ_D
	Obs. (1)	Exp. (2)	Bias (3)	Bias (4)	Bias (5)
<i>Panel A. HellaSwag, strict benchmark (20 models, 9 families, 6 developers)</i>					
OLS	0.406 (0.043)	0.430 (0.007)	−0.024 (0.076) [0.77]	0.105 (0.107) [0.37]	−0.090 (0.619) [0.89]
Year FE	0.460 (0.035)	0.437 (0.007)	0.024 (0.080) [0.78]	0.022 (0.147) [0.89]	0.025 (0.727) [0.97]
Developer FE	0.449 (0.014)	0.436 (0.008)	0.013 (0.044) [0.77]	0.066 (0.148) [0.68]	−0.030 (0.763) [0.97]
Family FE	0.473 (0.243)	0.464 (0.010)	0.008 (0.158) [0.96]	0.185 (0.046) [0.14]	−0.227 (0.116) [0.15]
Lab × period FE	0.476 (0.239)	0.440 (0.008)	0.037 (0.132) [0.79]	0.053 (0.149) [0.74]	0.023 (0.339) [0.95]
<i>Panel B. HellaSwag, benchmark with OLMo-2 (57 models, 30 families, 19 developers)</i>					
OLS	0.328 (0.046)	0.329 (0.007)	−0.001 (0.041) [0.98]	0.079 (0.091) [0.43]	−0.050 (0.070) [0.58]
Year FE	0.319 (0.076)	0.348 (0.007)	−0.029 (0.037) [0.47]	0.101 (0.107) [0.36]	−0.113 (0.212) [0.60]
Developer FE	0.393 (0.054)	0.370 (0.008)	0.024 (0.018) [0.66]	0.141 (0.070) [0.14]	−0.090 (0.098) [0.57]
Family FE	0.441 (0.048)	0.409 (0.013)	0.031 (0.064) [0.63]	0.203 (0.059) [0.048]	−0.262 (0.106) [0.027]
Lab × period FE	0.442 (0.032)	0.405 (0.010)	0.037 (0.039) [0.36]	0.165 (0.077) [0.13]	−0.113 (0.241) [0.65]
Notability-propensity control	0.335 (0.071)	0.350 (0.006)	−0.015 (0.043) [0.80]	0.076 (0.125) [0.55]	−0.057 (0.072) [0.53]
<i>Panel C. OLS, other outputs and supports</i>					
HellaSwag, strict surface, 57	0.328 (0.046)	0.314 (0.014)	0.014 (0.045) [0.76]	0.090 (0.091) [0.37]	−0.032 (0.073) [0.73]
ARC-Challenge, 20	0.466 (0.117)	0.473 (0.018)	−0.007 (0.248) [0.98]	0.218 (0.188) [0.50]	−0.123 (1.429) [0.93]
ARC-Challenge, 57, with OLMo-2	0.449 (0.055)	0.413 (0.027)	0.036 (0.069) [0.61]	0.171 (0.182) [0.49]	−0.045 (0.129) [0.74]
Winogrande, 20	0.371 (0.055)	0.550 (0.038)	−0.179 (0.263) [0.53]	−0.040 (0.165) [0.90]	−0.251 (1.341) [0.86]
Winogrande, 57, with OLMo-2	0.351 (0.047)	0.501 (0.044)	−0.150 (0.077) [0.084]	−0.027 (0.194) [0.89]	−0.225 (0.202) [0.28]

Note: Output: log-odds of above-chance accuracy. Obs.: estimator on the observed outputs of released base models. Exp.: the same estimator, with the same fixed effects and controls, on the outputs that the experimental technology produces for the same models: a quadratic in $(\ln N, \ln D)$ fitted to the OLMo ladder and [Gadre et al. \(2025\)](#) ($N \geq 0.1B$; 86 runs); Panel B adds the released OLMo-2 7B and 13B models (88 runs), as an earlier version of this analysis did. Each panel uses the models inside the convex hull of its own runs; ‘strict surface, 57’ evaluates the Panel A surface on the Panel B models, 37 of which lie outside its support. Bias = Obs. − Exp. Parentheses: CV3 jackknife s.e. by developer (Obs.; bias: of Obs. − Exp., with the benchmark’s s.e. added in quadrature) and the design-conditional wild-cluster bootstrap s.e. of the surface (9,999 draws; Exp.). Brackets: the largest of three few-cluster p -values for zero bias that include the benchmark’s uncertainty (CV3 with $t(G - 1)$; restricted wild-cluster bootstrap- t , Webb weights, 9,999 draws, studentized by CR1 or by CV3). Effective numbers of clusters: 1.2–3.8 of 6 in Panel A (no p -value is reliable there); 2.5–6.7 of 19 in Panel B. ARC-Challenge and Winogrande: Tobit surface censored at the floor (52 and 16 of 86 runs are at it), level matched to the observed outputs, counterfactual clipped at the same floor. Notability-propensity control (selection on observables): quadratic in the estimated propensity of Epoch notability; Panel A has no notable model. *Source:* Author’s calculations. Observational outputs: Open LLM Leaderboard v1 via [Ruan, Maddison and Hashimoto \(2024\)](#) and [Maia Polo et al. \(2025\)](#); parameter counts from Hugging Face; release dates from [Epoch AI \(2026\)](#). Experimental runs: [Bhagia et al. \(2025\)](#), [Team OLMo et al. \(2024\)](#) (Panel B only) and [Gadre et al. \(2025\)](#).

to +0.037 across five estimators), but with six clusters they carry no inferential weight.

This is a failure to reject in a joint test of low power, not evidence of unbiasedness. On 57 models the 95 percent interval for the least-squares bias, with $t(18)$ critical values, is $[-0.088, 0.086]$, about 26 percent of the benchmark on either side. The bias detectable with 80 percent power is 0.12, or 37 percent of the benchmark; on the strict support it is 0.27, or 62 percent. The test covers only models with at most 8.8 billion parameters, and its benchmark is the technology of the AI2 and OpenLM recipes.

Other benchmarks. ARC-Challenge and Winogrande are less informative, because many small designed runs score at or below chance: 52 of the 86 runs are at the floor of chance-adjusted accuracy on ARC-Challenge and 16 on Winogrande, against 2 on HellaSwag. A least-squares surface fitted to the clipped log-odds makes the observational compute elasticity on ARC-Challenge look attenuated, with a bias of -0.098 on 57 models. The attenuation does not survive modeling the censoring. We fit a Tobit surface censored at the floor, match its level to the observed outputs, and clip the counterfactual at the same floor; the bias is then $+0.036$ (0.069), with $p = 0.61$. The Tobit benchmark extrapolates a Gaussian latent surface below the floor for 60 percent of the runs, so we do not read it as showing that the attenuation was an artifact. Winogrande keeps a borderline attenuation: -0.150 (0.077) on 57 models, with p -values between 0.068 and 0.084 across the three procedures. On the strict support the Winogrande estimate is similar but uninformative (-0.179 , $p = 0.53$).

Two parts of the joint null are not tested. Observational outputs come from the leaderboard’s harness, whereas the designed runs were evaluated in the OLMES and LLM-foundry harnesses with other prompts and numbers of shots; we harmonize them only through the log-odds of chance-adjusted accuracy. And benchmark contamination, or classifier-filtered corpora that favor benchmark-like text (Dominguez-Olmedo, Dorner and Hardt, 2025), raise output at given inputs; family effects absorb such shifts only when they are common to a family.

E. Parameters versus Data

The split between parameters and data is not reliably estimated. With family fixed effects on the 57-model design, the observational parameter elasticity exceeds the benchmark by 0.203 (0.059) and the data elasticity falls short of it by 0.262 (0.106). The conservative p -values are 0.048 and 0.027, and dropping any one developer leaves both signs unchanged (Appendix Table D1). But the identifying variation is thin. The effective number of clusters is about 4 of 19, and two families supply 65 percent of the leverage for θ_D : Cerebras-GPT, trained at exactly 20 tokens per parameter, and Phi, trained on synthetic data. Under the strict benchmark the biases have similar size ($+0.185$ and -0.227) but conservative p -values of 0.14 and 0.15. Biases of opposite sign are also hard to reconcile with transmission: if wedges are unrelated to productivity at given compute,

transmission shifts both elasticities by the same amount (Section I.B). Opposite signs point instead to factor-biased recipe changes within families or to errors in D , which the benchmark cannot distinguish from bias. We read them as a within-family disagreement driven by two families, not as a finding.

Both the observational data and the experimental technology agree that $\theta_N > \theta_D$ on this design: 0.42 against 0.27 under the experimental technology on the 57 models, and $\theta_N - \theta_D = 0.21$ (0.08) in the observational data on all 128. Over-trained models have wedges above one, so their output elasticity with respect to parameters exceeds that with respect to data. A regression on $\ln C = \ln 6ND$ alone therefore imposes equal weights that the data reject, and the residual of such an equal-weight index misattributes $\frac{1}{2}(\theta_N - \theta_D)(\Delta n - \Delta d)$ to productivity (Corollary 1(iii)): as models shift further toward tokens, measured productivity understates progress.

F. How Large Could Transmission Bias Be?

Two exercises calibrate the size of the biases that the validation could have missed. In the first, we treat the 25 pretraining recipes of DataDecide as labs with real outcomes and simulate how they choose model size. Pooled least squares is then unbiased under exogenous budgets. It overstates θ_C by 0.044 when funding rises with productivity rank, and understates it by 0.18, or 39 percent, when each lab trains to a common capability target. Cross-family productivity dispersion in the observational panel is about five times that of the recipes, which scales the funding bias to about +0.2 and the target bias beyond -0.18 . The 95 percent interval of the matched bias excludes both. The data are therefore consistent with budgets roughly orthogonal to productivity, or with offsetting funding and release selection; the simulated rules are stylized.

The second exercise is a simulated industry in which the compute rule is known (Table 2; the design and all estimators are in Appendix C). Forty developers each produce six generations of models, release one to four models per generation, and choose compute by one of four rules. Pooled nonlinear least squares with generation effects overstates γ by 13 percent under funding and understates it by 30 percent under a capability target. With one flagship per developer and generation, the overstatement under funding is 66 percent, as Proposition 1(i) predicts for one model per lab. Developer fixed effects remove about two-thirds of the funding bias but little of it with flagships only, because productivity is persistent rather than fixed. Estimation within developer and generation, the observational analog of an IsoFLOP sweep, is unbiased under every rule, but it requires at least two released siblings. The specification most common in machine learning, a pooled fit with E free and no time effects, misses more than half of productivity growth under three of the four rules, mostly by attributing it to scale (Corollary 1(ii)) and partly to a downward-biased estimate of E . The calibration is illustrative; only signs and rankings should be read from it.

TABLE 2. PRODUCTION-FUNCTION ESTIMATORS IN A SIMULATED INDUSTRY

Compute rule:	All released models				Flagships only
	(a) Exogenous	(b) Funding	(c) Target	(d) Predetermined	(e) Funding
<i>Panel A. Frontier elasticity γ (truth 0.178)</i>					
Pooled Huber, E free	0.019 [0.031]	0.027 [0.036]	-0.004 [0.022]	0.025 [0.035]	0.117 [0.123]
Pooled NLS, generation effects	-0.001 [0.010]	0.023 [0.024]	-0.053 [0.056]	0.018 [0.019]	0.118 [0.121]
Developer effects	0.000 [0.006]	0.008 [0.009]	-0.022 [0.024]	0.004 [0.007]	0.113 [0.114]
Developer $\times$ generation effects	0.000 [0.002]	0.000 [0.003]	0.000 [0.002]	0.000 [0.002]	n.a.
<i>Panel B. Productivity growth per generation (truth 0.150)</i>					
Pooled Huber, E free	-0.081 [0.081]	-0.090 [0.090]	-0.036 [0.039]	-0.088 [0.088]	-0.145 [0.146]
Pooled NLS, generation effects	0.002 [0.012]	-0.024 [0.027]	0.054 [0.055]	-0.018 [0.021]	-0.130 [0.132]
Developer effects	0.001 [0.010]	-0.009 [0.013]	0.023 [0.025]	-0.003 [0.010]	-0.124 [0.125]
Developer $\times$ generation effects	0.001 [0.009]	0.000 [0.009]	0.000 [0.009]	0.001 [0.009]	n.a.

Note: Bias [root-mean-squared error] over 400 replications per column. Forty developers each produce six generations of models and release one to four models per generation, about eightfold apart in compute; productivity is Hicks-neutral, $\omega_{ft} = 0.15(t-1) + x_{ft}$ with x_{ft} a persistent AR(1) (coefficient 0.8, standard deviation 0.25). Compute rules: (a) exogenous budgets; (b) funding, log compute rises by $2x_{ft}$; (c) a common loss target reached by buying compute; (d) predetermined budgets, log compute rises by $2x_{f,t-1}$; (e) rule (b) with one model per developer and generation. Allocations include inference demand (mean log wedge 0.46) and allocation errors (standard deviation 0.2 in $\ln(D/N)$). The pooled Huber fit estimates E and has no time effects, as is common in machine-learning practice; the other estimators know E . Developer $\times$ generation fixed effects use only the menu of sizes within a developer's generation and are not defined with flagships only (n.a.). Productivity growth: mean change in generation means of $y - \hat{F}(n, d)$. Every estimator starts from data-based values. The calibration is illustrative; magnitudes scale with the assumed share of productivity-driven variation in compute. Appendix Table C1 reports all estimators and parameters.

Source: Author's simulations; replication code `code/analysis/m6_montecarlo` (entry point `run_v2.py`).

IV. Measurement of Compute and the Standard Remedies

Measurement error. Measurement error in compute is small, but showing it requires an independent measure. Compute in the panel is $C = 6ND$ from reported N and D , and Epoch’s compute figures for these models are mostly $6ND$ itself (operation counting) or developer-reported counts of the same kind. Only records of hardware time measure compute independently of the reported inputs. We rebuild compute from them as chip-hours times peak dense 16-bit throughput times a common utilization of 0.3. We do not use Epoch’s utilization field, which is back-calculated from Epoch’s own compute estimate in every record we checked. Across the 27 models with such records, from 12 families, $\ln(C_{\text{hw}}/6ND)$ has a standard deviation of 0.29, with 95 percent interval [0.20, 0.41] (Appendix Table D2). This bounds the error in $\ln 6ND$ from above, because it also contains differences in true utilization and errors in chip-hours. The implied reliability of $\ln C$ on the 57-model design is at least 0.98 pooled and 0.955 within families. Correcting for errors in variables raises the pooled compute elasticity by 0.007 and the within-family elasticity by 0.021, or by 0.042 at the upper end of the interval for the error standard deviation (Appendix Table D3).¹

Reverse regression. The reverse regression of $\ln C$ on output gives an inverse slope of 0.560. By construction this equals the forward slope, 0.328, divided by the forward R^2 of 0.585 (with family effects, $0.441/0.760 = 0.580$). Its distance from the same statistic computed on the experimental outputs, 0.352, therefore measures output variation not explained by compute (productivity dispersion and noise), not a bias. Under funding, the forward–reverse pair need not bracket the truth at all (Proposition 1(iii)).

Instruments, panels and proxies. The standard remedies for transmission bias cannot be used, for reasons that concern the data rather than the estimators. The price-performance of the best available data-center accelerator is a step function of calendar time. On the 57-model design its first-stage F is 0.90 (0.01 with developer fixed effects), and the Anderson–Rubin sets are unbounded. Applied to the experimental outputs, the same instrument recovers 0.012 against a design-matched elasticity of 0.33: the instrument fails, not the data. The price-performance of each model’s own training hardware is strong only on 10 models from 7 developers ($F = 14.3$), and it takes three values, so it acts as an indicator of vintage; its Anderson–Rubin set is $[-1.50, 0.34]$. An export-control instrument (Chinese developers after October 2023, with developer and year effects) is strong among the 90 models with at most 14 billion parameters and 5 trillion tokens ($F = 10.6$). It gives $\theta_C = 0.90$ (0.25), with Anderson–Rubin set $[0.49, 2.23]$, which is 1.7 to 2.3 times the least-squares and fixed-effects estimates

¹An earlier version of this analysis reported a within-family errors-in-variables estimate of 0.858 against a benchmark of 0.409. It used Epoch’s ‘Confident’ uncertainty class ($\sigma_u = \ln 3/1.645 = 0.67$), which describes the uncertainty of Epoch’s own estimate rather than the error in $6ND$, and it computed the within-family variance of $\ln C$ with an $n - 1$ denominator, ignoring the 30 family means. With the correct variance and the hardware-time bound, the within-family bias is +0.052 (0.049), with $p = 0.30$.

on that sample. Its exclusion restriction fails because recipes and data changed at the same time. Dynamic-panel estimators (Arellano and Bond, 1991; Blundell and Bond, 1998) need three consecutive generations of a product line, which only six product-line cells have. Proxy inversion is unavailable for the reasons of Proposition 3. The notability-propensity control barely moves the matched bias (-0.015 , $p = 0.80$).

V. Productivity Dispersion

How dispersed is productivity across model families? The answer depends on the units in which it is measured (Table 3). Let Δ be the gap between the 90th and 10th percentiles of family productivity in HellaSwag log-odds, net of inputs. In output units the 90–10 ratio of the odds, e^Δ , is 1.9 to 3.0 across samples and netting methods. In reducible-loss units, $e^{\gamma\Delta/\theta_C}$ with the frontier elasticity $\gamma = 0.178$ of Besiroglu et al. (2024), it is 1.3 to 1.8. The 90–10 ratio within US manufacturing industries, 1.92 (Syverson, 2004), lies at the bottom of the first range and just above the second. In compute-equivalent units, e^{Δ/θ_C} , the compute that the 10th-percentile family needs to match the 90th, the ratio is 7 to 24 with the 57-model benchmark $\theta_C = 0.33$, or 4 to 13 with the strict benchmark’s 0.43. Within developers, in the design of Mertens, Fischl-Lanzoni and Thompson (2026), it is 11.1 in compute units, against their 41 on another benchmark.

Manufacturing’s ratio is both an output-unit and an input-unit ratio, because returns to scale are near one. For language models the two differ by the factor $1/\theta_C$ in logs, between 2.3 and 3, because returns to compute are small. The comparison with manufacturing is therefore a choice of cardinalization, not a finding.

Three caveats apply. First, family effects absorb benchmark contamination and the similarity of training data to the benchmark, as well as productivity. Second, training compute omits the compute of teachers and synthetic-data generators. With inputs netted within families, the synthetic-data families Phi and SmolLM rank first and third of 38, so measured productivity credits the student with an intermediate input, the gross-output problem of Gandhi, Navarro and Rivers (2020). Third, the conversion to compute rests on a single cardinal elasticity from one benchmark; the intervals in Table 3 draw θ_C from its bootstrap. Transmission, finally, would make all of these ratios understate dispersion (Proposition 1(ii)).

TABLE 3. PRODUCTIVITY DISPERSION ACROSS MODEL FAMILIES IN OUTPUT AND INPUT UNITS

	Output units odds	Compute $\theta_C = 0.33^a$	Loss	Compute $\theta_C = 0.43$	Loss	θ used
	(1)	(2)	(3)	(4)	(5)	(6)
<i>All 128 models (38 families)</i>						
Family effects, θ_C netting	2.68 [1.94, 5.27]	19.9 [7.5, 160.7]	1.70 [1.43, 2.47]	13.1 [5.1, 69.8]	1.58 [1.34, 2.13]	0.33
Family effects, within netting	2.83 [2.02, 5.59]	23.7 [8.5, 193.1]	1.76 [1.46, 2.56]	11.2 [5.1, 53.7]	1.54 [1.34, 2.03]	0.33
Within-developer residuals	2.78 [1.77, 3.19]	11.1 [4.1, 18.1]	1.54 [1.29, 1.68]			0.43
<i>Without distilled and synthetic-data models</i>						
Family effects, θ_C netting	2.57 [1.90, 5.13]	17.6 [7.0, 143.9]	1.67 [1.42, 2.43]	13.1 [5.1, 58.9]	1.58 [1.34, 2.07]	0.33
Family effects, within netting	2.60 [1.87, 5.90]	18.3 [6.8, 216.6]	1.68 [1.41, 2.61]	9.2 [4.3, 58.9]	1.49 [1.30, 2.07]	0.33
Within-developer residuals	2.84 [1.78, 3.21]	11.3 [4.0, 18.0]	1.54 [1.28, 1.67]			0.43
<i>Also without code families (32 families)</i>						
Family effects, θ_C netting	2.07 [1.70, 2.29]	9.1 [4.9, 12.7]	1.48 [1.33, 1.57]	6.9 [3.5, 10.4]	1.41 [1.25, 1.52]	0.33
Family effects, within netting	1.90 [1.61, 5.10]	7.0 [4.2, 143.4]	1.41 [1.29, 2.42]	4.4 [3.3, 37.3]	1.30 [1.23, 1.91]	0.33
Within-developer residuals	2.51 [1.71, 3.04]	8.4 [3.8, 14.2]	1.46 [1.27, 1.61]			0.43
Manufacturing (Syverson, 2004)	1.92	1.92	1.92	1.92	1.92	≈ 1

Note: 90–10 ratios of Hicks-neutral productivity on HellaSwag log-odds q . A gap Δ between the 90th and 10th percentiles is reported as an odds ratio e^Δ (column 1, output units), as compute-equivalents e^{Δ/θ_C} (input units: the compute the 10th-percentile family needs to match the 90th), and in reducible-loss units $e^{\gamma\Delta/\theta_C}$ with $\gamma = 0.178$ (Besiroglu et al., 2024); $\gamma = 0.155$ (Hoffmann et al., 2022) is in the replication files. $\theta_C = 0.33$: the design-matched benchmark on 57 models (benchmark including OLMo-2); $\theta_C = 0.43$: the strictly experimental benchmark on 20 models (Table 1). Family effects net inputs either with the benchmark θ_C or with the within-family (θ_N, θ_D); within-developer residuals follow Mertens, Fischl-Lanzoni and Thompson (2026) (developer and year effects, single-model developers dropped) and use the regression’s own θ_C (column 6) in columns 2–3 (^a). Brackets: 90 percent intervals from a developer-cluster bootstrap (999 draws) that also draws θ_C from its benchmark bootstrap. Manufacturing’s 1.92 is an output-unit ratio and, with returns to scale near one, also an input-equivalent ratio; for LLMs the two differ by the factor $1/\theta_C$ in logs, because returns to compute are small. Family effects also absorb benchmark contamination, similarity of training data to the benchmark and omitted teacher compute (distillation, synthetic data).

Source: Author’s calculations; outputs from Ruan, Maddison and Hashimoto (2024) and Maia Polo et al. (2025).

VI. Algorithmic Progress

Ho et al. (2024) estimate that the compute needed to reach a given perplexity halves every 8.4 months. Their data and code are public, and their preferred model is the technology (1) with $E = 0$, $\kappa = 1$ and factor-augmenting productivity growing linearly in time, $\psi_N = g_N t$ and $\psi_D = g_D t$:

$$\ln \text{ppl} = \exp(\alpha_{0b} - \alpha g_N t - \alpha \ln N) + \exp(\beta_{0b} - \beta g_D t - \beta \ln D),$$

where b indexes three benchmarks and t is time in years. Effective compute grows at $g_C = g_N + g_D$ per year and doubles every $\tau_C = 12 \ln 2 / g_C$ months; Hicks neutrality is $\alpha g_N = \beta g_D$. Their appendix plots the joint bootstrap distribution of the two year coefficients; we ask what its shape implies for the doubling time.

A. The Ridge

The rate is weakly identified because the sample is not on the expansion path, a problem of the kind studied by Diamond, McFadden and Rodriguez (1978). To first order the data pin down how fast loss falls at fixed inputs, an average of αg_N and βg_D weighted by the mean shares of the two terms in loss (0.55 and 0.45). The doubling time instead weights the two rates by $1/\alpha$ and $1/\beta$. The two combinations coincide only on the expansion path, along which parameters would grow with compute at elasticity $a = 0.37$ under their exponents. Off the path, pairs of rates that fit equally well imply different doubling times.

Figure 3 shows the resulting ridge. The bootstrap correlation between αg_N and βg_D is -0.73 with the authors' code and -0.82 at the converged estimates discussed below. The profile-likelihood interval for τ_C is $[4.1, 40.5]$ months. The parameter-augmenting share g_N/g_C can lie anywhere in $[-0.44, 3.37]$ (95 percent profile interval), and over that range τ_C runs from 5.3 to 39.6 months (Table 4). These likelihood ratios assume independent Gaussian errors, although observations are clustered by paper. The profile over g_N/g_C also has a second local minimum near -0.28 ; a missed global minimum could only widen the interval.

B. Neutrality

Precision about the rate is bought with an assumption about the bias. Hicks neutrality is not rejected (bootstrap $p = 0.29$ and 0.94 for the converged penalized and the unpenalized estimators), which reflects low power rather than evidence of neutrality. Imposing it gives 9.2 months with unpenalized least squares, with interval $[6.3, 14.3]$, or 11.6 months with the converged penalized estimator: the tightest intervals among the specifications that the data do not reject. As Diamond, McFadden and Rodriguez (1978) predict, the rate is identified here by the restriction on the bias. Even that restriction is fragile. If the true technology has $E > 0$, neutrality imposed within the $E = 0$ form is misspecified. In simulations

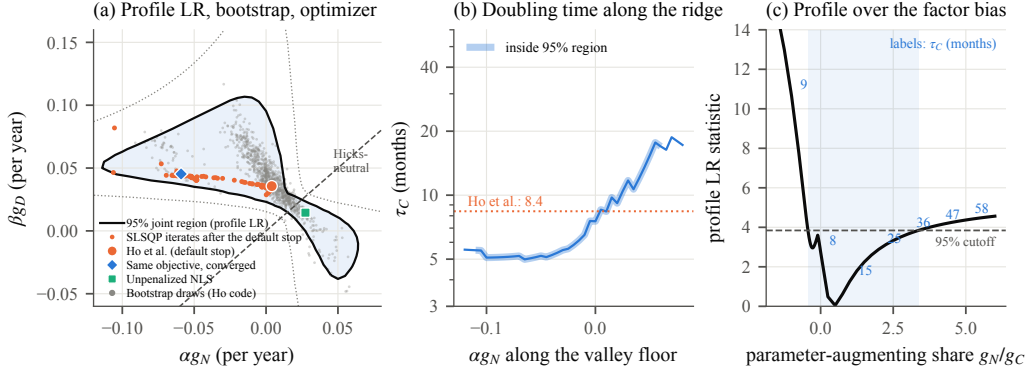


FIGURE 3. THE RIDGE IN ESTIMATES OF ALGORITHMIC PROGRESS

Note: Data of Ho et al. (2024), 231 model-benchmark observations. Panel (a): 95 percent joint profile-likelihood region for the year terms (α_N, β_D) , bootstrap draws from the authors' code, the optimizer's iterates from its default stopping point to the converged minimum of the same objective, the unpenalized least-squares minimum, and the Hicks-neutral line. Panel (b): doubling time of effective compute τ_C along the floor of the profile; the thick segment lies inside the 95 percent region. Panel (c): profile likelihood-ratio statistic over the parameter-augmenting share g_N/g_C , with the implied τ_C in months at selected points; the dashed line is the 95 percent cutoff and the shaded band the 95 percent interval, $[-0.44, 3.37]$. Likelihood ratios assume independent Gaussian errors.

on Ho et al.'s design with a true doubling time of 12 months, it yields 4.2 to 4.4 months, so the neutral estimates may overstate progress.

C. Replication and the Published Estimate

We reproduce Ho et al.'s estimates exactly from their data and code: the same value of their objective (0.0518), every parameter within 4×10^{-7} , and a bootstrap median of 8.44 months under their protocol (Table 4, Panel A). The published 8.4 months is that median; the point estimate is 8.7. The estimator minimizes mean squared error plus an L_1 penalty, using SciPy's SLSQP routine with its default stopping rule, which halts after 18 iterations. On a flat ridge this setting matters. Run to convergence from the same start, the objective falls from 0.0518 to 0.0507 while mean squared error changes by only 0.27 percent, and all 300 random starts end below the published value. Almost all of the improvement comes from the penalty, on constants whose values depend on an arbitrary normalization. The converged point has $\tau_C = 6.1$ months, with a paper-cluster interval of $[3.0, 22.7]$, about twice as wide as published. Unpenalized least squares gives 10.2 months, with interval $[4.1, 27.6]$. Estimators of the same model thus give 6.1 to 10.2 months, and the data cannot arbitrate between them. We read this as a consequence of the ridge, not as a correction of the published rate: on a flat objective, the stopping rule and the penalty select the point. The finding concerns numerical optimization, not the order of magnitude of progress: every point estimate of this model lies between half a year and a year per doubling. It

TABLE 4. ALGORITHMIC PROGRESS IN THE DATA OF HO ET AL. (2024): REPLICATION AND RIDGE

	τ_C (months)	95% interval	MSE	Objective
<i>Panel A. Replication of Ho et al.'s preferred model (231 observations, 144 papers)</i>				
Published (bootstrap median)	8.4	[4.5, 14.3]	–	0.0518129
Their code and protocol, median	8.44	[4.4, 14.1]	0.0464	0.0518129
point estimate, cluster interval	8.68	[4.1, 16.9]		
Same objective, run to convergence	6.08	[3.0, 22.7]	0.0463	0.0507227
share of 300 starts below published				1.00
Unpenalized least squares (NLS)	10.2	[4.1, 27.6]	0.0453	
profile-likelihood interval		[4.1, 40.5]		
Benchmark-specific E_b estimated	10.5	[2.9, 26.3]	0.0451	
<i>Panel B. The ridge: what the data identify</i>				
Corr(α_N, β_D), their code / converged	–0.73 / –0.82			
Share g_N/g_C , 95% profile interval	[–0.44, 3.37]			
range of τ_C over that interval	5.3–39.6			
Hicks neutrality p : converged / NLS	0.29 / 0.94			
Hicks neutrality imposed, converged	11.6	[6.6, 17.6]	0.0465	
Hicks neutrality imposed, NLS	9.2	[6.3, 14.3]	0.0454	
Effective data (converged)	25.7	[3.9, ∞)	0.0484	
Exponents of Besiroglu et al. (2024) imposed	$g_C < 0$	[72.5, ∞)	0.1126	
Year terms $\div$ exponents (Whitfill)	76.5	[46.2, ∞)	–	

Note: Ho et al.'s model: $\ln \text{ppl} = \exp(\alpha_{0b} - \alpha g_N t - \alpha \ln N) + \exp(\beta_{0b} - \beta g_D t - \beta \ln D)$, irreducible loss zero, t in years; $\tau_C = 12 \ln 2 / (g_N + g_D)$ is the doubling time of effective compute in months. Their estimator minimizes $\text{MSE} + 0.0025 \sum |\theta|$ by SLSQP from zero with scipy's default tolerance, which stops after 18 iterations; 'converged' runs the same objective and algorithm to convergence. The objective gap (0.0518 against 0.0507) is almost all penalty: MSE changes by 0.27 percent, and the penalty falls on constants whose values depend on an arbitrary normalization. Intervals: paper-cluster bootstrap (400 draws), percentiles of $g_N + g_D$ mapped to τ_C (∞ when non-positive growth lies inside); the first two rows use Ho et al.'s iid bootstrap. Profile intervals invert an iid Gaussian likelihood ratio and are indicative, because papers are clustered. The interval for g_N/g_C is located on a grid of step 0.025 near its boundaries (an earlier version's coarser grid gave [–0.39, 3.37], and its range of 5 to 38 months was computed over the grid points [–0.25, 3.25]); the profile has a second local minimum near –0.28, and a missed global minimum could only widen the interval. Hicks neutrality: $\alpha g_N = \beta g_D$. Effective data: D measured as effective data under repetition (Muennighoff et al., 2023).

Source: Author's calculations from the data and code of Ho et al. (2024).

parallels the finding of Besiroglu et al. (2024) for the third approach of Hoffmann et al. (2022), and convergence problems familiar from demand estimation (Dubé, Fox and Su, 2012; Knittel and Metaxoglou, 2014).

D. Why Is the Data Exponent Small?

Ho et al.’s data exponent is 0.04, against about 0.37 in experiments, and Whitfill (2025) reads the gap as evidence that progress is overstated about ninefold: dividing their year coefficients by experimental exponents gives 76.5 months. Imposing experimental exponents in estimation is rejected (mean squared error rises by a factor of 1.3 to 4.8 across 14 experimental technologies) and implies negative or implausibly slow progress. Two specification choices explain much of the exponent gap while moving τ_C much less (Appendix Table D4). First, fitting the $E = 0$ form to the Chinchilla sweep itself lowers the frontier elasticity from 0.178 to 0.053, close to the sweep’s mean elasticity of total rather than reducible loss (0.054). The like-for-like gap is therefore about 2 to 2.5-fold, not 7 to 9-fold. Dropping E rescales exponents and year coefficients by similar factors, so rates barely move: imposing a common E from 0 to 1.5 nats per word doubles $\hat{\gamma}$ but moves τ_C only from 10.2 to 12.4 months. Ho et al.’s data themselves prefer $E = 0$ within this form, so this argument rests on the sweep and on simulations; in the simulations the authors’ procedure can also stop at a local optimum that overstates progress. Second, their D is dataset size, while small models were trained for a median of 62 to 140 epochs. Measuring effective data under repetition (Muennighoff et al., 2023) raises τ_C by a factor of 1.5 to 4, with an unbounded upper confidence limit.

E. Scale Dependence and Frontier Regressions

Two further questions need designed data. Whether the compute-equivalent gain of transformers over earlier architectures depends on scale turns on whether their frontier elasticities differ (Corollary 1(i)). In Ho et al.’s data the difference is 0.008, with interval $[-0.012, 0.034]$, and bootstrap intervals for the implied gains span six orders of magnitude (Appendix Table D5). And regressions of loss on compute that omit time effects overstate the compute elasticity by a factor of 1.15 on all 231 observations and 1.44 among 36 record-setting releases (Corollary 1(ii)). Among record setters, whose compute moves closely with time, independently measured growth rates imply an algorithmic share of 0.37 and a predicted factor of 1.58, consistent with the estimate. On all observations the prediction fails, because compute and time are only moderately correlated (0.62), and the ordinary omitted-variable formula applies instead.

F. The Missing Data

Proposition 4 names the data that would settle these questions: IsoFLOP sweeps at several vintages on a common output, whose drift in the compute-

optimal M^* signs the bias of technical change and whose curvature fixes its size. The controlled experiment of Okar (2026) supplies the cross-sectional analog for a change in data quality at one date. [TBD-m9: one sentence on the shift in the compute-optimal M^* at fixed compute between the two corpora of the experiment, and hence the sign of the Hicks bias of the data-quality difference (Proposition 4(i) applied across corpora rather than dates); delete if m9 cannot answer it.]

VII. Allocative versus Technical Change

Measured progress, the fall in the compute needed to reach a given loss, mixes shifts of the frontier with movements toward it. By Proposition 5, the move from Kaplan-rule to Chinchilla-rule allocations changed the mix, not the technology, so its gain is allocative. Gundlach et al. (2025) define the compute-equivalent gain from such rebalancing and count it as algorithmic progress.

Measurement. Measuring the gain realized between eras requires a choice. Training-cost efficiency, the Farrell efficiency against the training-only frontier (Farrell, 1957), $C_{\min}(L(N, D))/6ND$, counts deliberate over-training as waste, although over-training reflects the value that labs place on compact models. Serving demand cannot produce a wedge below one (Section I.A). We therefore count only the elimination of under-training: training-cost efficiency evaluated at $\min(w, 1)$, which by Lemma 1(iii) depends on the wedge alone. The truncated measure attributes every wedge below one to allocative error. A wedge below one can also reveal a binding data constraint or data-augmenting productivity specific to the lab. For 2020–21 models trained on a few hundred billion tokens, a belief in the law of Kaplan et al. (2020) is the more plausible reading, but it remains an assumption.

Realized gains. We compare models in the Epoch database with at least 10^{23} FLOP: 7 released in 2020–21 (era 1) and 92 in 2022–24 (era 2) (Table 5, Figure 4). Every era-1 model has $w < 1$ under every technology, and the median ratio of tokens to parameters rose from 1.7 to 92 between the eras. Counting only the elimination of under-training, geometric-mean training-cost efficiency rose by a factor of 2.0 under the reference technology of Okar (2026) (Chinchilla with κ free) and 1.7 under Besiroglu et al. (2024), with 95 percent interval $[1.21, 2.44]$. Across all 14 estimated technologies the factor ranges from 1.1 to 4.9. Era-2 releases closed 80 to 94 percent of the era-1 under-training gap, in logs. The positive sign is close to mechanical, because every era-1 model is under-trained and truncation can only raise era-2 efficiency; the content is in the size. Without truncation the gain ranges from 0.55 to 4.3 across technologies: counting over-training as waste can reverse its sign.

The intervals resample models within era; only the Besiroglu interval also draws the technology’s parameters, and with seven era-1 models all intervals are rough. Technologies estimated on other sweeps use their own conventions for N , D and the loss corpus, so their levels are indicative. The Marin fits, whose parameter

TABLE 5. ALLOCATIVE GAINS FROM REBALANCING, WITH AND WITHOUT TRUNCATING THE WEDGE AT ONE

	$w < 1$ (%)	Untruncated		Truncated		Gap	Kaplan
	era 1/2	Gain	95% CI	Gain	95% CI	closed	CEG
	(1)	(2)	(3)	(4)	(5)	(6)	(7)
<i>Chinchilla sweep</i>							
Besiroglu et al. (2024)	100 / 18	1.05	[0.57, 1.75]	1.68	[1.21, 2.44]	0.92	11.7
Hoffmann et al. (2022)	100 / 45	1.91	[1.53, 2.50]	2.34	[1.87, 3.03]	0.88	32.4
Hoffmann et al. (2022), rounded	100 / 50	2.41	[1.86, 3.24]	2.77	[2.16, 3.69]	0.86	46.7
Chinchilla, κ free (reference)	100 / 26	1.19	[0.92, 1.56]	1.97	[1.58, 2.53]	0.92	20.3
<i>Other public sweeps, Chinchilla form (Okar, 2026)</i>							
Farseer, total N	100 / 42	1.58	[1.27, 2.04]	2.06	[1.66, 2.60]	0.89	38.7
Farseer, non-embedding N	100 / 18	1.03	[0.86, 1.25]	1.54	[1.32, 1.84]	0.92	7.3
Gadre et al. (2025), C4	100 / 12	0.60	[0.51, 0.70]	1.14	[1.06, 1.26]	0.94	3.5
OLMo ladder	100 / 66	2.52	[1.99, 3.20]	2.63	[2.07, 3.33]	0.81	57.0
Muennighoff et al. (2023)	100 / 38	1.74	[1.38, 2.27]	2.33	[1.87, 2.97]	0.90	18.1
<i>Laboratory IsoFLOP designs, Chinchilla form (Okar, 2026, Online Appendix)</i>							
Llama 3 IsoFLOPs (Meta)	100 / 12	0.74	[0.62, 0.90]	1.30	[1.16, 1.50]	0.93	5.1
Marin, Comma	100 / 62	4.31	[2.95, 6.60]	4.95	[3.42, 7.50]	0.80	388.5
Marin, DCLM	100 / 50	3.04	[2.09, 4.45]	4.00	[2.83, 5.77]	0.85	230.7
Marin, Nemotron-CC	100 / 49	2.57	[1.87, 3.68]	3.52	[2.55, 5.11]	0.86	166.3
(Mis)Fitting, FineWeb/C4	100 / 12	0.55	[0.40, 0.75]	1.64	[1.31, 2.12]	0.93	15.0
<i>Cross-lab regression</i>							
Ho et al. (2024), model 7	0 / 0	0.09	[0.07, 0.12]	1.00	[1.00, 1.00]	—	—
<i>Other samples, Besiroglu et al. (2024) technology</i>							
$C \geq 10^{23}$, cleaned (7, 79)	100 / 19	1.09	[0.60, 1.78]	1.70	[1.23, 2.43]	0.94	
Top five per year (10, 15)	70 / 40	1.27	[0.91, 1.72]	1.31	[1.05, 1.76]	0.73	
Epoch frontier models (7, 6)	86 / 33	1.56	[1.07, 2.19]	1.54	[1.12, 2.19]	0.82	

Note: Models in the Epoch database with $C \geq 10^{23}$ FLOP: 7 released in 2020–21 (era 1) and 92 in 2022–24 (era 2); dense, at most four epochs, $D = C/6N$. Training-cost efficiency, the Farrell efficiency against the training-only frontier, $CE = C_{\min}(L(N, D))/6ND$, depends only on the wedge $w = \varepsilon_N/\varepsilon_D$ within the Chinchilla family. Untruncated gain: ratio of era geometric means of CE . Truncated gain: the same with $CE(\min(w, 1))$, which counts only the elimination of under-training; over-training ($w > 1$) can reflect the value of compactness, including serving demand, and is not treated as inefficiency, whereas serving demand cannot produce $w < 1$ (Okar, 2026, Proposition 1). Column 1: share of models with $w < 1$. Column 6: $\ln(\text{truncated gain})$ divided by the era-1 log compute lost to under-training. Column 7: compute-equivalent gain of abandoning the Kaplan rule ($N \propto C^{0.73}$ from GPT-3) at 5×10^{26} FLOP, the counterfactual of Gundlach et al. (2025); the Kaplan allocations have $w < 1$ under every technology, so truncation does not change it. Intervals: percentile bootstrap over models within era (999 draws); only the Besiroglu et al. (2024) row also draws the technology’s parameters, so the other intervals hold the technology fixed, and with seven era-1 models all intervals are rough. A wedge below one is counted as allocative error; by Okar (2026, Proposition 1) it can also reflect a binding data constraint or data-augmenting productivity specific to the developer. Technologies from other sweeps use each sweep’s own N and D conventions and loss corpus, so their levels are indicative. Group labels give the data on which each technology is estimated: the Chinchilla sweep (the reference technology of Okar, 2026 is Chinchilla with κ free), Chinchilla-form fits to the other public sweeps and to laboratory IsoFLOP designs, and Ho et al.’s cross-lab model, under which every model has $w > 1$.

Source: Author’s calculations from the Epoch AI models database (Epoch AI, 2026).

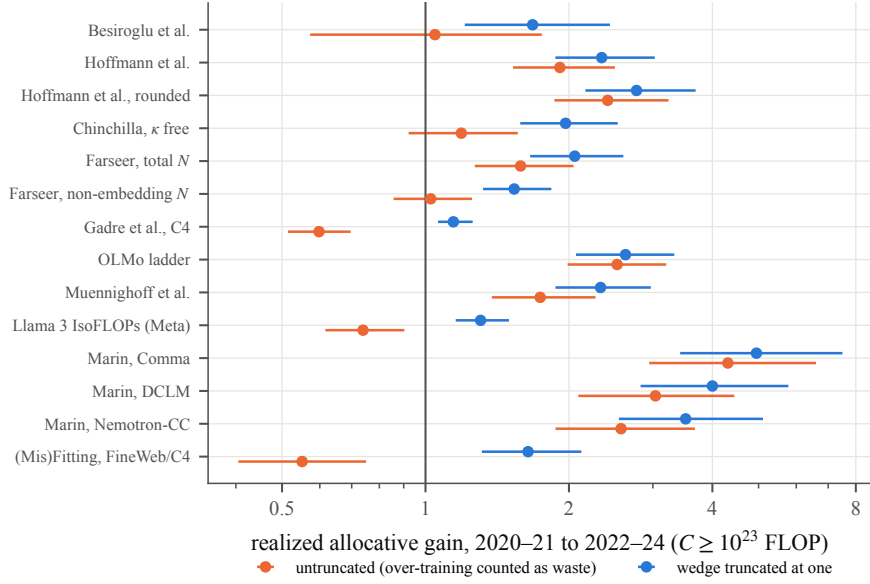


FIGURE 4. REALIZED ALLOCATIVE GAINS FROM REBALANCING, BY TECHNOLOGY

Note: Ratio of geometric-mean training-cost efficiency, 2022–24 against 2020–21, for Epoch models with at least 10^{23} FLOP (7 and 92 models), under each technology, with 95 percent percentile-bootstrap intervals over models within era (log scale). Orange: efficiency against the training-only frontier, which counts over-training as waste. Blue: the wedge truncated at one, which counts only the elimination of under-training. Only the Besiroglu et al. (2024) interval also reflects uncertainty in the technology's parameters. Estimates in Table 5.

exponents are about twice Chinchilla’s, give the largest gains (3.5 to 4.9). Samples of the five largest models per year and of Epoch’s frontier models give smaller and noisier truncated gains (1.3 and 1.5 under Besiroglu).

Comparison with progress. Against the effective-compute gain implied by Ho et al.’s published rate over the same interval, 8.9-fold over 2.28 years, the realized allocative gain amounts to 6 to 73 percent in logs across technologies (31 percent under the reference technology). The denominator itself ranges from 6.4 to 22.5 across the estimators of Section VI. Gundlach et al. (2025) report that the Kaplan-to-Chinchilla rebalancing is worth about a factor of 10 at the 2025 frontier, and our counterfactual agrees. Under the Besiroglu technology, continuing Kaplan’s rule from GPT-3 would cost a factor of 1.6 at GPT-3’s compute, 5.2 at 3.8×10^{25} FLOP and 11.7 at 5×10^{26} FLOP. Because Kaplan-rule allocations have $w < 1$ under every technology, truncation does not affect this counterfactual. What we add is the contrast with the gain realized between eras, 1.1 to 4.9, which is smaller. Under Ho et al.’s own exponents, finally, every model has $w > 1$. An attenuated data exponent makes tokens nearly worthless, so a regression with those exponents cannot credit rebalancing to inputs and loads it onto the time trend.

VIII. Conclusion

Observational data on language models deliver returns to compute once the design is matched, though only as a failure to reject in a test of low power, and measures of productivity dispersion whose size depends on the units. They do not deliver the split between parameters and data, the bias of technical change, or its rate without a restriction on the bias. In each case the missing variation is the variation transverse to the expansion path that designed IsoFLOP sweeps supply: at given compute, labs choose the mix, and productivity does not move it.

Three implications follow for economists who use these numbers. First, cross-lab regressions of benchmark scores on compute can be used for returns to compute once the design is matched to an experimental benchmark, but not for the relative value of parameters and data. Second, headline rates of algorithmic progress from observational data carry the precision of an identifying assumption about the bias of technical change; forecasts and growth accounts that use them should carry intervals that span that assumption. Third, rebalancing gains realized in the past are small relative to counterfactual gains at today’s scale, so the importance of allocation for measured progress depends on which of the two a question needs.

What would settle these questions is designed variation published by labs: IsoFLOP sweeps at several dates on a common output, and model-level records of hardware time. Both cost little relative to the frontier runs they would inform, and the second is already partly public. A final lesson is methodological. The industry allows an economist to check production-function estimators against

experiments, and the check works only after the design is matched; where experiments exist in other industries, the same discipline applies.

REFERENCES

- Akerberg, Daniel A., Kevin Caves, and Garth Frazer.** 2015. “Identification Properties of Recent Production Function Estimators.” *Econometrica*, 83(6): 2411–2451.
- Arellano, Manuel, and Stephen Bond.** 1991. “Some Tests of Specification for Panel Data: Monte Carlo Evidence and an Application to Employment Equations.” *The Review of Economic Studies*, 58(2): 277–297.
- Bagnoli, Mark, and Ted Bergstrom.** 2005. “Log-Concave Probability and Its Applications.” *Economic Theory*, 26(2): 445–469.
- Besiroglu, Tamay, Ege Erdil, Matthew Barnett, and Josh You.** 2024. “Chinchilla Scaling: A Replication Attempt.” *arXiv preprint arXiv:2404.10102*.
- Bhagia, Akshita, Jiacheng Liu, Alexander Wettig, David Heineman, Oyvind Tafjord, Ananya Harsh Jha, Luca Soldaini, Noah A. Smith, Dirk Groeneveld, Pang Wei Koh, Jesse Dodge, and Hannaneh Hajishirzi.** 2025. “Establishing Task Scaling Laws via Compute-Efficient Model Ladders.” *Second Conference on Language Modeling (COLM)*.
- Blundell, Richard, and Stephen Bond.** 1998. “Initial Conditions and Moment Restrictions in Dynamic Panel Data Models.” *Journal of Econometrics*, 87(1): 115–143.
- Cameron, A. Colin, and Douglas L. Miller.** 2015. “A Practitioner’s Guide to Cluster-Robust Inference.” *Journal of Human Resources*, 50(2): 317–372.
- Cameron, A. Colin, Jonah B. Gelbach, and Douglas L. Miller.** 2008. “Bootstrap-Based Improvements for Inference with Clustered Errors.” *Review of Economics and Statistics*, 90(3): 414–427.
- Carter, Andrew V., Kevin T. Schnepel, and Douglas G. Steigerwald.** 2017. “Asymptotic Behavior of a t-Test Robust to Cluster Heterogeneity.” *Review of Economics and Statistics*, 99(4): 698–709.
- Dehejia, Rajeev H., and Sadek Wahba.** 1999. “Causal Effects in Nonexperimental Studies: Reevaluating the Evaluation of Training Programs.” *Journal of the American Statistical Association*, 94(448): 1053–1062.
- Diamond, Peter A., Daniel McFadden, and Miguel Rodriguez.** 1978. “Measurement of the Elasticity of Factor Substitution and Bias of Technical Change.” In *Production Economics: A Dual Approach to Theory and Applications*. Vol. 2, ed. Melvyn Fuss and Daniel McFadden, 125–147. Amsterdam:North-Holland.

- Dominguez-Olmedo, Ricardo, Florian E. Dorner, and Moritz Hardt.** 2025. “Training on the Test Task Confounds Evaluation and Emergence.” *International Conference on Learning Representations*.
- Dubé, Jean-Pierre, Jeremy T. Fox, and Che-Lin Su.** 2012. “Improving the Numerical Performance of Static and Dynamic Aggregate Discrete Choice Random Coefficients Demand Estimation.” *Econometrica*, 80(5): 2231–2267.
- Epoch AI.** 2026. “Data on AI Models.” *Epoch AI database, snapshot of 23 September 2026*, <https://epoch.ai/data/ai-models>.
- Erdil, Ege, and Tamay Besiroglu.** 2022. “Algorithmic Progress in Computer Vision.” *arXiv preprint arXiv:2212.05153*.
- Farrell, M. J.** 1957. “The Measurement of Productive Efficiency.” *Journal of the Royal Statistical Society. Series A (General)*, 120(3): 253–290.
- Gadre, Samir Yitzhak, Georgios Smyrnis, Vaishaal Shankar, Suchin Gururangan, Mitchell Wortsman, Rulin Shao, Jean Mercat, Alex Fang, Jeffrey Li, Sedrick Keh, Rui Xin, Marianna Nezhurina, Igor Vasiljevic, Luca Soldaini, Jenia Jitsev, Alexandros G. Dimakis, Gabriel Ilharco, Pang Wei Koh, Shuran Song, Thomas Kollar, Yair Carmon, Achal Dave, Reinhard Heckel, Niklas Muennighoff, and Ludwig Schmidt.** 2025. “Language Models Scale Reliably with Over-Training and on Downstream Tasks.” *International Conference on Learning Representations*.
- Gandhi, Amit, Salvador Navarro, and David A. Rivers.** 2020. “On the Identification of Gross Output Production Functions.” *Journal of Political Economy*, 128(8): 2973–3016.
- Goldberger, Arthur S.** 1981. “Linear Regression after Selection.” *Journal of Econometrics*, 15(3): 357–366.
- Griliches, Zvi, and Jacques Mairesse.** 1998. “Production Functions: The Search for Identification.” In *Econometrics and Economic Theory in the 20th Century: The Ragnar Frisch Centennial Symposium*. *Econometric Society Monographs*, ed. Steinar Strøm, Chapter 6, 169–203. Cambridge:Cambridge University Press.
- Gundlach, Hans, Alex Fogelson, Jayson Lynch, Ana Trišović, Jonathan Rosenfeld, Anmol Sandhu, and Neil Thompson.** 2025. “On the Origin of Algorithmic Progress in AI.” *arXiv preprint arXiv:2511.21622*.
- Hall, Robert E.** 1988. “The Relation between Price and Marginal Cost in U.S. Industry.” *Journal of Political Economy*, 96(5): 921–947.

- Hausman, Jerry A., and David A. Wise.** 1977. "Social Experimentation, Truncated Distributions, and Efficient Estimation." *Econometrica*, 45(4): 919–938.
- Heckman, James J.** 1979. "Sample Selection Bias as a Specification Error." *Econometrica*, 47(1): 153–161.
- Ho, Anson, Tamay Besiroglu, Ege Erdil, David Owen, Robi Rahman, Zifan Carl Guo, David Atkinson, Neil Thompson, and Jaime Sevilla.** 2024. "Algorithmic Progress in Language Models." *Advances in Neural Information Processing Systems*, 37: 58245–58283.
- Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre.** 2022. "Training Compute-Optimal Large Language Models." *Advances in Neural Information Processing Systems*, 35: 30016–30030.
- Hotz, V. Joseph, Guido W. Imbens, and Julie H. Mortimer.** 2005. "Predicting the Efficacy of Future Training Programs Using Past Experiences at Other Locations." *Journal of Econometrics*, 125(1–2): 241–270.
- Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei.** 2020. "Scaling Laws for Neural Language Models." *arXiv preprint arXiv:2001.08361*.
- Klepper, Steven, and Edward E. Leamer.** 1984. "Consistent Sets of Estimates for Regressions with Errors in All Variables." *Econometrica*, 52(1): 163–183.
- Knittel, Christopher R., and Konstantinos Metaxoglou.** 2014. "Estimation of Random-Coefficient Demand Models: Two Empiricists' Perspective." *Review of Economics and Statistics*, 96(1): 34–59.
- König, Gunnar, Martin Pawelczyk, Ulrike von Luxburg, and Sebastian Bordt.** 2026. "Validity Threats for Foundation Model Research." *arXiv preprint arXiv:2606.05029*.
- LaLonde, Robert J.** 1986. "Evaluating the Econometric Evaluations of Training Programs with Experimental Data." *American Economic Review*, 76(4): 604–620.
- Levinsohn, James, and Amil Petrin.** 2003. "Estimating Production Functions Using Inputs to Control for Unobservables." *Review of Economic Studies*, 70(2): 317–341.

- León-Ledesma, Miguel A., Peter McAdam, and Alpo Willman.** 2010. “Identifying the Elasticity of Substitution with Biased Technical Change.” *American Economic Review*, 100(4): 1330–1357.
- MacKinnon, James G., and Matthew D. Webb.** 2018. “The Wild Bootstrap for Few (Treated) Clusters.” *The Econometrics Journal*, 21(2): 114–135.
- MacKinnon, James G., Morten Ørregaard Nielsen, and Matthew D. Webb.** 2023. “Cluster-Robust Inference: A Guide to Empirical Practice.” *Journal of Econometrics*, 232(2): 272–299.
- Magnusson, Ian, Nguyen Tai, Ben Bogin, David Heineman, Jena D. Hwang, Luca Soldaini, Akshita Bhagia, Jiacheng Liu, Dirk Groeneveld, Oyvind Tafjord, Noah A. Smith, Pang Wei Koh, and Jesse Dodge.** 2025. “DataDecide: How to Predict Best Pretraining Data with Small Experiments.” *Proceedings of the 42nd International Conference on Machine Learning*, 267: 42487–42502.
- Maia Polo, Felipe, Seamus Somerstep, Leshem Choshen, Yuekai Sun, and Mikhail Yurochkin.** 2025. “Sloth: Scaling Laws for LLM Skills to Predict Multi-Benchmark Performance Across Families.” *Advances in Neural Information Processing Systems*, 38.
- Marschak, Jacob, and William H. Andrews, Jr.** 1944. “Random Simultaneous Equations and the Theory of Production.” *Econometrica*, 12(3/4): 143–205.
- Mertens, Matthias, Natalia Fischl-Lanzoni, and Neil Thompson.** 2026. “Is There “Secret Sauce” in Large Language Model Development?” *arXiv preprint arXiv:2602.07238*.
- Muennighoff, Niklas, Alexander M. Rush, Boaz Barak, Teven Le Scao, Nouamane Tazi, Aleksandra Piktus, Sampo Pyysalo, Thomas Wolf, and Colin Raffel.** 2023. “Scaling Data-Constrained Language Models.” *Advances in Neural Information Processing Systems*, 36: 50358–50376.
- Nagy, Béla, J. Doyne Farmer, Quan M. Bui, and Jessika E. Trancik.** 2013. “Statistical Basis for Predicting Technological Progress.” *PLoS ONE*, 8(2): e52669.
- Okar, Yigit.** 2026. “What Optimizing Labs Reveal: Scaling Laws as Production Functions.” *Working Paper, Care AI*.
- Olley, G. Steven, and Ariel Pakes.** 1996. “The Dynamics of Productivity in the Telecommunications Equipment Industry.” *Econometrica*, 64(6): 1263–1297.
- Owen, David.** 2024. “How Predictable Is Language Model Benchmark Performance?” *arXiv preprint arXiv:2401.04757*.

- Ruan, Yangjun, Chris J. Maddison, and Tatsunori Hashimoto.** 2024. “Observational Scaling Laws and the Predictability of Language Model Performance.” *Advances in Neural Information Processing Systems*, 37: 15841–15892.
- Sevilla, Jaime, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos.** 2022. “Compute Trends Across Three Eras of Machine Learning.” *2022 International Joint Conference on Neural Networks (IJCNN)*, 1–8.
- Syverson, Chad.** 2004. “Product Substitutability and Productivity Dispersion.” *Review of Economics and Statistics*, 86(2): 534–550.
- Syverson, Chad.** 2011. “What Determines Productivity?” *Journal of Economic Literature*, 49(2): 326–365.
- Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, et al.** 2024. “2 OLMo 2 Furious.” *arXiv preprint arXiv:2501.00656*.
- Todd, Petra E., and Kenneth I. Wolpin.** 2006. “Assessing the Impact of a School Subsidy Program in Mexico: Using a Social Experiment to Validate a Dynamic Behavioral Model of Child Schooling and Fertility.” *American Economic Review*, 96(5): 1384–1417.
- Webb, Matthew D.** 2023. “Reworking Wild Bootstrap-Based Inference for Clustered Errors.” *Canadian Journal of Economics*, 56(3): 839–858.
- Whitfill, Parker.** 2025. “Note on Selection Bias in Observational Estimates of Algorithmic Progress.” *arXiv preprint arXiv:2508.11033*.
- Zellers, Rowan, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi.** 2019. “HellaSwag: Can a Machine Really Finish Your Sentence?” *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 4791–4800.

APPENDIX A. PROOFS

Proof of Lemma 1. (i) Write effective inputs $\tilde{n} = n + \psi_N$ and $\tilde{d} = d + \psi_D$, $u = Ae^{-\alpha\tilde{n}}$ and $v = Be^{-\beta\tilde{d}}$. Then $y = \omega - \kappa \ln(u+v) - \zeta$ and $\tilde{n} + \tilde{d} = \tilde{c} - \ln 6$ with $\tilde{c} \equiv c + \psi_N + \psi_D$. The first-order condition for the split is $\alpha u = \beta v$, that is $w = 1$; it is sufficient because $\ln(u+v)$ is convex in $(\tilde{n}, \tilde{d})$ and the constraint is linear. Taking logs, $\alpha\tilde{n} - \beta\tilde{d} = (\alpha + \beta) \ln G$; with $\tilde{d} = \tilde{c} - \ln 6 - \tilde{n}$ this gives $\tilde{n}^* = \ln G + a(\tilde{c} - \ln 6)$. Subtracting ψ_N , $n^* = \ln G + a(c - \ln 6) + a\psi_D - (1-a)\psi_N = \ln G + a(c - \ln 6) + \chi/(\alpha + \beta)$, and ω does not appear. At the optimum $u + v = u(\alpha + \beta)/\beta$ and $u = AG^{-\alpha}e^{-\alpha a(\tilde{c} - \ln 6)}$, with $\kappa\alpha a = \gamma$. Hence $y^* = \omega + \gamma(\tilde{c} - \ln 6) - \ln K - \zeta$ with $K \equiv [(\alpha + \beta)AG^{-\alpha}/\beta]^\kappa$, and $\omega + \gamma(\psi_N + \psi_D) = \Omega$.

(ii) At given compute write $\tilde{n} = \tilde{n}^* - x/2$ and $\tilde{d} = \tilde{d}^* + x/2$, where $x = \ln(\tilde{M}/\tilde{M}^*) = \ln(M/M^*(C))$ because effective and actual ratios differ by the same factor $e^{\psi_D - \psi_N}$. Then $\ln w = \ln(\alpha A/\beta B) - \alpha\tilde{n} + \beta\tilde{d}$, which is zero at the optimum and changes by $(\alpha + \beta)x/2$.

(iii) Let $Q = u + v$ at the run. Because $R = e^{-\omega + \zeta}Q^\kappa$, the cheapest bundle attaining R is the cheapest attaining Q in effective units, and effective compute is proportional to actual compute at given productivity. The cheapest bundle has $\alpha u^* = \beta v^*$ and $u^* + v^* = Q$, so $u^* = aQ$ and $v^* = (1-a)Q$. Since $u \propto N^{-\alpha}$ and $v \propto D^{-\beta}$, $C/C_{\min} = (u^*/u)^{1/\alpha}(v^*/v)^{1/\beta}$. With $s_u \equiv u/Q = \beta w/(\alpha + \beta w)$, $u^*/u = (\alpha + \beta w)/[(\alpha + \beta)w]$ and $v^*/v = (\alpha + \beta w)/(\alpha + \beta)$; using $1/\alpha + 1/\beta = (\alpha + \beta)/(\alpha\beta)$ gives (2), in which A, B, E, κ and productivity do not appear. For the expansion, let $x = \ln w$. Then $\ln[(\alpha + \beta e^x)/(\alpha + \beta)] = \ln(1 - a + ae^x) = ax + \frac{1}{2}a(1-a)x^2 + O(x^3)$. Multiplying by $(\alpha + \beta)/(\alpha\beta) = 1/[(\alpha + \beta)a(1-a)]$ and subtracting $x/\alpha = x/[(\alpha + \beta)(1-a)]$ leaves $x^2/[2(\alpha + \beta)]$. $\square$

Proof of Proposition 1. (i) $\text{Cov}(y, c) = \gamma \text{Var}(c) + \pi_1 V_\omega$ and $\text{Var}(c) = \pi_1^2 V_\omega + V_\eta$. Substituting $\pi_1 = -1/\gamma$ gives the target-rule expression. If also $V_\eta = 0$, then $y = \omega + \gamma(\pi_0 - \omega/\gamma) - \zeta = \gamma\pi_0 - \zeta$ up to a constant, which does not depend on c . (ii) The residual is $y - b_f c = \omega V_\eta / \text{Var}(c) - \eta \pi_1 V_\omega / \text{Var}(c) - \zeta$ up to a constant. Its variance is $V_\omega V_\eta (V_\eta + \pi_1^2 V_\omega) / \text{Var}(c)^2 + V_\zeta = V_\omega V_\eta / \text{Var}(c) + V_\zeta$, and $V_\eta / \text{Var}(c) < 1$ when $\pi_1 \neq 0$ and $V_\omega > 0$. (iii) $\text{Var}(y) - \gamma \text{Cov}(c, y) = V_\omega + 2\gamma\pi_1 V_\omega + \gamma^2 \text{Var}(c) + V_\zeta - \gamma\pi_1 V_\omega - \gamma^2 \text{Var}(c) = (1 + \gamma\pi_1)V_\omega + V_\zeta$. With $\text{Cov}(c, y) > 0$, $b_f \leq \gamma$ if and only if $\pi_1 \leq 0$, and $\gamma_r \geq \gamma$ if and only if $(1 + \gamma\pi_1)V_\omega + V_\zeta \geq 0$. Under funding both inequalities give estimates above γ . (iv) By Lemma 1(i), on the path $n = \ln G + a(c - \ln 6) + \chi/(\alpha + \beta)$ exactly, and ω does not enter. $\square$

Proof of Proposition 2. (i) With $\pi_1 = 0$, c is independent of S , and a run is released if and only if $S \geq \bar{y} - \gamma c$ (up to a constant). Let $m(k) \equiv \mathbb{E}[S \mid S \geq k]$. Then $\mathbb{E}[y \mid c, \text{released}] = \gamma c + m(\bar{y} - \gamma c)$. The function m is nondecreasing, because for $k_1 < k_2$, $\mathbb{E}[S \mid S \geq k_1]$ is a mixture of $\mathbb{E}[S \mid k_1 \leq S < k_2] \leq k_2$ and $\mathbb{E}[S \mid S \geq k_2] \geq k_2$. So the conditional mean is γc plus a nonincreasing function of c , and the linear projection of a nonincreasing function of c on c has a nonpositive slope. If S has a log-concave density, the mean residual life $m(k) - k$ is nonincreasing (Bagnoli and Bergstrom, 2005), so $0 \leq m' \leq 1$, the derivative of the conditional mean lies in $[0, \gamma]$, and so does the projection slope, which is a

weighted average of that derivative with nonnegative weights.

(ii) Under joint normality $c = \mathbb{E}c + \delta(y - \mathbb{E}y) + \nu$ with ν independent of y and $\text{Var}(\nu) = V_{c|y}$. Selection on y changes neither the distribution of ν nor its independence of y , so in the released sample $\text{Cov}(c, y) = \delta \text{Var}_{\text{sel}}(y)$ and $\text{Var}_{\text{sel}}(c) = \delta^2 \text{Var}_{\text{sel}}(y) + V_{c|y}$, with $\text{Var}_{\text{sel}}(y) = r \text{Var}(y)$ given by the truncated-normal variance. The forward slope is the stated b_{sel} ; its derivative in r has the sign of $\delta V_{c|y}$. If $\pi_1 \leq 0$ and $\delta > 0$, then $b_{\text{sel}} \leq b_f$ (the value at $r = 1$) and $b_f \leq \gamma$ by Proposition 1(i). If $\pi_1 > 0$, then $\delta > 0$, and $\text{Var}(y) - \gamma \text{Cov}(c, y) = (1 + \gamma\pi_1)V_\omega + V_\zeta > 0$ gives $\gamma\delta < 1$; rearranging $b_{\text{sel}} > \gamma$ gives $r > r^*$. (iii) Selection lowers the forward slope and leaves $\gamma_r = 1/\delta$ unchanged; combine with Proposition 1(iii). $\square$

Proof of Proposition 3. (i) An econometrician who ignores factor-augmenting productivity computes $\hat{w} = \alpha A e^{-\alpha n} / (\beta B e^{-\beta d})$, and the lab's true wedge is $w = \alpha A e^{-\alpha(n+\psi_N)} / (\beta B e^{-\beta(d+\psi_D)}) = \hat{w} e^\chi$. By Lemma 1(ii) with $\chi = 0$, $\ln \hat{w} = \frac{\alpha+\beta}{2} [\ln M - m_0(c)]$, where $m_0(c) = d^* - n^* = -2 \ln G + (1 - 2a)(c - \ln 6)$ is the compute-optimal log ratio of Lemma 1(i) with $\chi = 0$. Setting $w = \bar{w}$ and solving for $\ln M$ gives the stated expression, in which ω does not appear. (ii) $n = \frac{1}{2}(c - \ln 6 - \ln M)$ and $d = \frac{1}{2}(c - \ln 6 + \ln M)$, so every function of (n, d) , including F , is a function of $(c, \ln M)$. Under a strictly monotone rule $c = h(\omega, z)$ and $\chi = 0$, Lemma 1(i) gives n and d as functions of c , so $y = \omega + F(n(c), d(c)) - \zeta$, and a control function in c for ω also absorbs $F(n(c), d(c))$. $\square$

Proof of Proposition 4. (i) At given compute the optimum solves $r(n, d, t) \equiv \ln(F_n/F_d) = 0$ with $n + d = c - \ln 6$. Differentiating, $dn = -dd$ and $(r_d - r_n) dd + r_t dt = 0$. At the optimum $F_n = F_d$, so $r_d - r_n = (2F_{nd} - F_{nn} - F_{dd})/F_n$, which is positive by the second-order condition for a maximum along the isocost line (strict at a regular interior optimum). The isoquant through the optimum is tangent to the isocost there, so the derivative of r along the direction $(-1, 1)$ is the same along both. Along the isoquant, the level marginal rate of technical substitution is $(F_n/F_d)M$, and the elasticity of substitution $\sigma = d \ln M / d \ln \text{MRTS}$ implies $dr = (1/\sigma - 1) d \ln M$; with $d \ln M = 2 dd$, $r_d - r_n = 2(1/\sigma^* - 1)$. Positivity gives $\sigma^* \in (0, 1)$, and $d \ln M^* / dt = 2 dd / dt = -2r_t / (r_d - r_n) = -r_t \sigma^* / (1 - \sigma^*)$ with $r_t = \mathcal{B}_t$.

(ii) With $\psi_{N,t} = g_N t$ and $\psi_{D,t} = g_D t$, $\mathcal{B} = \partial_t \ln(\alpha u / \beta v) = \beta g_D - \alpha g_N$ at fixed inputs. By Lemma 1(i), $\ln M^* = d^* - n^* = -2 \ln G + (1 - 2a)(c - \ln 6) - 2\chi_t / (\alpha + \beta)$, which drifts at $-2(\beta g_D - \alpha g_N) / (\alpha + \beta) = 2[(1 - a)g_N - a g_D]$, and y^* shifts at $\gamma(g_N + g_D)$. Given a and γ , the two drifts are linear in (g_N, g_D) with determinant -1 , so (g_N, g_D) are identified. For non-identification of $S \equiv \alpha + \beta$, fix (a, γ) and, for any $S > 0$, set $\alpha_S = (1 - a)S$, $\beta_S = aS$, $\kappa_S = \gamma / [a(1 - a)S]$ and choose (A_S, B_S) to reproduce G and K . Each member has the same expansion path $n^*(c)$ and frontier $y^*(c)$ at every date, because with $(A_{S,t}, B_{S,t}) = (A_S e^{-\alpha_S g_N t}, B_S e^{-\beta_S g_D t})$ the path intercept drifts at $a g_D - (1 - a)g_N$ and the frontier at $\gamma(g_N + g_D)$, neither of which involves S . The bias $\mathcal{B} = S[a g_D - (1 - a)g_N]$ scales with S . (iii) Follows from the drift of $\ln M^*$ in (ii). $\square$

Proof of Proposition 5. By Lemma 1(i), a lab with (ω, ψ_N, ψ_D) needs compute

$6[Ke^{-\Omega}/R]^{1/\gamma}$ to reach reducible loss R on its own expansion path, where R excludes noise. Hence $\ln C_{\min,F}(L_i) = \ln 6 + [\ln K - \Omega_F - \ln(L_i - E)]/\gamma$, with $\ln(L_i - E) = -\omega_i + \zeta_i + \ln Q_i^\kappa$. The run's own minimal compute for its noise-free loss is $\ln C_{\min,i} = \ln 6 + [\ln K - \Omega_i + \omega_i - \ln Q_i^\kappa]/\gamma$, and $\ln(6N_i D_i) - \ln C_{\min,i} = \Phi(w_i)$ by Lemma 1(iii). Subtracting gives $\ln C_{\min,i} - \ln C_{\min,F}(L_i) = (\Omega_F - \Omega_i + \zeta_i)/\gamma$. The Gopher figure applies (2) with the parameters of Besiroglu et al. (2024) at $N = 280\text{B}$ and $D = 300\text{B}$. $\square$

Proof of Corollary 1. (i) The cost function of the old technology is $C_{\text{old}}^*(L) = 6[K_{\text{old}}/(L - E_{\text{old}})]^{1/\gamma_{\text{old}}}$. With $x = C/6$, $\ln f = \gamma_{\text{old}}^{-1}[\ln K_{\text{old}} - \ln(E_{\text{new}} - E_{\text{old}} + K_{\text{new}}x^{-\gamma_{\text{new}}})] - \ln x$, so $d \ln f / d \ln x = (\gamma_{\text{new}}/\gamma_{\text{old}})K_{\text{new}}x^{-\gamma_{\text{new}}}/(E_{\text{new}} - E_{\text{old}} + K_{\text{new}}x^{-\gamma_{\text{new}}}) - 1$, which vanishes for all x if and only if $E_{\text{new}} = E_{\text{old}}$ and $\gamma_{\text{new}} = \gamma_{\text{old}}$. (ii) $\text{Cov}(-\ln R_t, c_t) = \gamma \text{Var}(c) + \gamma g_A \text{Cov}(t, c)$ and $\text{Cov}(t, c) = g \text{Var}(t)$, so the slope is $\gamma[1 + (g_A/g)g^2 \text{Var}(t)/\text{Var}(c)]$. With $\text{Var}(u) = 0$ the R^2 is one and the slope is $\gamma(g + g_A)/g = \gamma/(1 - s_A)$. (iii) Subtract $\bar{\varepsilon}(\Delta n + \Delta d)$ from the first-order expansion. For discrete changes the error is second order. $\square$

APPENDIX B. DATA

Released models. The panel combines the base-model file of Ruan, Maddison and Hashimoto (2024) with the data of Maia Polo et al. (2025), which add base models absent from the first source and fill missing scores. Five rows of the second source that duplicate the first source's scores for the deduplicated Pythia models are dropped. Parameter counts are the floating-point tensor counts of the Hugging Face checkpoints where available; counting all stored tensors would include causal-mask buffers, which inflates the counts of 17 models. Reported token counts are corrected where a model card or paper gives a different figure, and every correction is listed with its source in the replication package. The outputs are HellaSwag (10-shot), ARC-Challenge (25-shot) and Winogrande (5-shot) accuracies from the same evaluation harness, mapped to $q = \text{logit}[(\text{acc} - \text{chance})/(1 - \text{chance})]$ with chance-adjusted accuracy clipped to $[0.01, 0.99]$; the composite is the logit of the mean chance-adjusted accuracy of the three tasks. The main sample contains 128 dense models with N , D and all three outputs, from 38 families and 22 developers, released between March 2021 and July 2024; the overlap sample ($N \leq 14\text{B}$, $D \leq 5\text{T}$) has 90 models. Compute is $C = 6ND$ in every row by construction. Within families the standard deviation of $\ln D$ is 0.42, against 1.29 for $\ln N$; only 18 families (66 models) vary D at all, and the correlation of $\ln N$ and $\ln D$ is 0.46.

Designed runs. The experimental benchmark uses the OLMo ladder and the runs of Gadre et al. (2025) with at least 0.1 billion parameters; the released OLMo-2 models, whose inputs their developer chose, enter only the alternative benchmark. The designed runs were evaluated with their designers' own harnesses, so the correspondence between output formats is part of the maintained null hypothesis of Section III. The semi-synthetic exercise uses the final DataDecide checkpoints with at least 60 million parameters (7 sizes, 25 recipes and 3 seeds).

Algorithmic progress. We re-implement line by line the sample construction of [Ho et al. \(2024\)](#) from their public notebooks, starting from the 408-row sheet. We keep rows with numeric N and D , not marked as excluded, outliers or uncertain, and with at least one perplexity. We stack the rows by benchmark, drop four named systems (μ P and LoRA fine-tunes), and keep the three best-performing models per paper. This yields 231 observations from 144 papers: 103 on WikiText-103, 80 on Penn Treebank and 48 on WikiText-2. D is the dataset size in unique tokens, not tokens processed. Epochs are known for 155 of the 231 observations; we impute the rest by the median within bins of $\log_{10} D$ and set epochs to one for $D \geq 10^{10}$. The effective-data variant follows [Muennighoff et al. \(2023\)](#). The output is log perplexity, mostly per word; its units vary with each evaluation’s tokenization. The data sheet carries no explicit license (the authors’ code is MIT-licensed), and we use it for estimation only.

Large models. For the allocative analysis we use the Epoch AI database ([Epoch AI, 2026](#)). Of the 792 language models with N and C , 544 report a dataset size and 454 are not fine-tunes. Of these, 428 are dense. In 339 of them, $C/(6N)$ lies within a factor of three of dataset size times epochs, and 247 of those were trained for at most four epochs. We set $D = C/(6N)$ so that $6ND = C$ exactly.

APPENDIX C. SIMULATIONS

C1. A Simulated Industry

Setup. Forty labs each produce six generations of models. In each generation a lab releases one to four models whose compute budgets are roughly eightfold apart. Productivity is Hicks-neutral and equals $\omega_{ft} = 0.15(t-1) + x_{ft}$, where the lab component follows $x_{ft} = 0.8x_{f,t-1} + \xi_{ft}$ with stationary standard deviation 0.25; seed noise has standard deviation 0.05 in $y = -\ln(L-E)$. The dispersion of ω is assumed, chosen to match the 41-fold within-developer 90–10 ratio in effective compute of [Mertens, Fischl-Lanzoni and Thompson \(2026\)](#); productivity growth of 0.15 per generation is also assumed. A lab-generation’s log budget is $c_{ft} = \bar{c}_t + h_f - p_{ft} + \nu_{ft}$, where $\bar{c}_t$ grows by $\ln 3$ per generation from 10^{21} FLOP, h_f is lab size (standard deviation 0.7), p_{ft} is an observed compute-price shifter (0.4), and ν_{ft} is noise (0.4). Four compute rules map productivity into budgets: (a) exogenous budgets; (b) funding that responds to current productivity ($+2x_{ft}$); (c) a loss target that the lab reaches by buying compute; and (d) compute provisioned one generation ahead ($+2x_{f,t-1}$), the timing assumption of [Akerberg, Caves and Frazer \(2015\)](#). Labs minimize lifetime compute given inference demand $T = 3D^*(c)\theta$ with $\ln \theta \sim N(0, 0.8^2)$, so that the mean log wedge is 0.46, and make allocation errors with standard deviation 0.2 in $\ln(D/N)$. Variants add release selection (about 18 percent of trained models unreleased), classical measurement error in $\ln D$ (standard deviation 0.15), data-augmenting lab heterogeneity, and a flagship-only sample. The estimators, applied to released models, are listed in Table C1. E is known to all of them except the pooled Huber fit that mimics

current practice in machine learning. Every estimator starts from data-based values; relative to an earlier version of this analysis, whose least-squares estimators started at the truth, the biases of $\hat{\gamma}$ and of productivity growth change by at most 0.003. Under the loss-target rule the pooled criterion has several local optima, and the bias of the pooled estimate of a moves from -0.03 to -0.06 .

Transmission bias is diluted but not removed by common fixes. Pooled non-linear least squares with generation effects overstates γ by 0.023 (13 percent) under funding and 0.018 under predetermined budgets, and understates it by 0.053 (30 percent) under a loss target (Table C1, Panel A). With one model per lab-generation under funding the bias is +0.118, matching the closed form of Proposition 1(i) applied to this design (0.118). Lab fixed effects remove about two-thirds of the bias under funding and four-fifths under predetermined budgets, but not under the target rule (-0.022), and almost none of it with flagships only (+0.113), because ω is persistent rather than fixed. The bias loads on the frontier elasticity, not on the allocation: the regression of $\ln N$ on $\ln C$ recovers a with a bias of at most 0.003 in absolute value in every scenario (Panel B, IV row), as Proposition 1(iv) predicts.

Within-family variation is the observational analog of an IsoFLOP experiment. Lab $\times$ generation fixed effects, which use only the size menu within a lab-generation, are unbiased for γ , a and productivity growth under every compute rule (absolute bias at most 0.002), because siblings share ω and their tier and inference-demand differences are exogenous. The estimator needs at least two released siblings, is unavailable with flagships only, and is hurt by measurement error in D (bias in $\ln M^*$ of -0.33 at an error standard deviation of 0.15, similar to lab fixed effects, -0.28). Estimators in the spirit of Akerberg, Caves and Frazer (2015) remove the transmission bias when the timing assumption is right (absolute bias at most 0.002 for γ with lagged instruments only) and fail when it is wrong (-0.027 under the target rule if current compute is treated as predetermined). Both variants, however, use the current within-family deviations of inputs as instruments, without which the lagged moment set is under-identified, so they borrow the same within-family variation; the lagged-instrument version is also about five times less precise for a (RMSE 0.12 to 0.13 against 0.025 to 0.028). The simulation therefore does not show that proxy or timing estimators work on flagship-only panels.

First-order-condition estimators mistake inference demand for technology. A system that stacks the within-family loss equation with the training-only first-order condition ($w = 1$) leaves the exponents essentially unbiased but biases $\ln M^*$ by +1.25 to +1.28 under every compute rule (Panel D), overstating M^* about 3.5-fold. This is close to the closed form $2 \mathbb{E}[\ln w]/(\alpha + \beta) = 2 \times 0.457/0.714 = 1.28$ implied by Lemma 1(ii): the residual of the first-order condition is the wedge, and ignoring it makes over-trained models look like evidence that data are cheap. A noisy usage proxy for T (log noise 0.3) removes the bias (-0.02 to -0.03 , RMSE at most 0.05). The instrumental-variable estimator that reads M^* off the observed

path inherits the same contamination (median bias +1.27 to +1.32), although its median bias for γ is small (-0.002 to $+0.005$; median absolute error 0.027 to 0.031) when releases are not selected.

Current practice absorbs more than half of productivity growth into the scaling law. The pooled Huber fit with E free and no time effects, the specification common in the machine-learning literature, understates productivity growth by 0.081 to 0.090 of a true 0.150 per generation under rules (a), (b) and (d) (Panel C), and overstates $\ln M^*$ by 0.53 to 0.64. Holding E at the truth isolates the scale-attribution channel of Corollary 1(ii): the same pooled regression still misses 36 to 43 percent of productivity growth and steepens the frontier by 0.050 to 0.059. Roughly two-thirds of the absorbed progress is attributed to scale and one-third to a downward-biased asymptote $\hat{E}$.

Selection and its correction interact with transmission. With exogenous budgets and a notability threshold for release, pooled estimates understate γ by 0.008 and overstate productivity growth by 0.011; a Heckman (1979) correction, which assumes that the inputs of unreleased models are observed and that a release-policy shifter is excluded, roughly halves both. Under predetermined budgets, selection (−) and transmission (+) offset (net +0.01), and the correction raises the bias to +0.018 by removing only the selection part. Selection also invalidates the compute-price instrument (median bias -0.022 and -0.033). The industry calibration is illustrative, not estimated; only signs and rankings should be read from it, and the magnitudes scale with the assumed ratio of productivity-driven to total variance in compute.

C2. Verification of the Transmission and Selection Formulas

Table C2 checks the transmission-bias formula of Proposition 1(i) against simulations with 10^6 draws per case. The simulated slopes match the closed form to within 10^{-3} in all twelve cases, on the expansion path and with lifetime-optimal allocations. The sign of the bias follows the compute rule: exogenous budgets give γ , funding and predetermined budgets overstate it, and a common loss target drives the slope to zero. The forward and reverse regressions bracket γ under exogenous budgets and dispersed targets, but under funding and predetermined budgets both exceed it. In samples of 240 lab-generations the nominal 95 percent least-squares interval covers γ in 95 percent of samples under exogenous budgets, but in 53 percent under mild funding, at most 4 percent under stronger funding, and 16 percent under predetermined budgets with $\lambda = 1$; these coverage rates depend on the assumed exogenous dispersion of compute. The residual dispersion also behaves as predicted by Proposition 1(ii): under funding with $\lambda = 2$ the standard deviation of the residual is 0.229, against 0.255 for productivity plus noise.

Outcome-based release biases the compute slope toward zero (Proposition 2). With exogenous budgets and an absolute release threshold, the slope falls from 0.178 to 0.112, 0.082 and 0.057 as 25, 50 and 75 percent of runs are dropped.

TABLE C1. PRODUCTION-FUNCTION ESTIMATORS IN A SIMULATED INDUSTRY: ALL ESTIMATORS

Compute rule:	(a) Exogenous	(b) Funding	(c) Target	(d) Predetermined
<i>Panel A. Frontier elasticity γ (truth 0.178)</i>				
Pooled Huber, E free (ML practice)	0.019 [0.031]	0.027 [0.036]	-0.004 [0.022]	0.025 [0.035]
Pooled NLS + generation FE	-0.001 [0.010]	0.023 [0.024]	-0.053 [0.056]	0.018 [0.019]
Lab FE	0.000 [0.006]	0.008 [0.009]	-0.022 [0.024]	0.004 [0.007]
Lab $\times$ generation FE	0.000 [0.002]	0.000 [0.003]	0.000 [0.002]	0.000 [0.002]
ACF, current c as instrument	0.000 [0.010]	0.007 [0.009]	-0.027 [0.034]	0.000 [0.010]
ACF, lagged instruments only	-0.001 [0.011]	-0.001 [0.011]	-0.002 [0.011]	-0.001 [0.011]
IV (compute price) + path [‡]	-0.002 [0.031]	0.005 [0.031]	-0.002 [0.027]	-0.002 [0.028]
FOC system, T ignored	-0.002 [0.003]	-0.002 [0.003]	-0.003 [0.003]	-0.002 [0.003]
FOC system, T proxied	-0.001 [0.001]	-0.001 [0.001]	-0.001 [0.001]	-0.001 [0.001]
<i>Panel B. Allocation exponent a (truth 0.513)</i>				
Pooled Huber, E free (ML practice)	-0.006 [0.111]	-0.004 [0.115]	-0.004 [0.082]	-0.010 [0.108]
Pooled NLS + generation FE	-0.002 [0.085]	0.004 [0.042]	-0.060 [0.191]	0.000 [0.045]
Lab FE	0.001 [0.066]	0.001 [0.043]	-0.010 [0.112]	0.001 [0.060]
Lab $\times$ generation FE	0.000 [0.027]	0.001 [0.028]	0.000 [0.025]	0.002 [0.027]
ACF, current c as instrument	0.004 [0.117]	0.001 [0.054]	-0.055 [0.241]	0.009 [0.121]
ACF, lagged instruments only	-0.001 [0.126]	-0.001 [0.121]	0.000 [0.126]	0.003 [0.130]
IV (compute price) + path [‡]	0.000 [0.003]	0.000 [0.002]	-0.003 [0.004]	0.000 [0.002]
FOC system, T ignored	-0.002 [0.004]	-0.002 [0.004]	-0.004 [0.005]	-0.002 [0.004]
FOC system, T proxied	-0.001 [0.003]	-0.001 [0.003]	-0.001 [0.003]	-0.001 [0.003]
<i>Panel C. TFP growth per generation (truth 0.150)</i>				
Pooled Huber, E free (ML practice)	-0.081 [0.081]	-0.090 [0.090]	-0.036 [0.039]	-0.088 [0.088]
Pooled NLS + generation FE	0.002 [0.012]	-0.024 [0.027]	0.054 [0.055]	-0.018 [0.021]
Lab FE	0.001 [0.010]	-0.009 [0.013]	0.023 [0.025]	-0.003 [0.010]
Lab $\times$ generation FE	0.001 [0.009]	0.000 [0.009]	0.000 [0.009]	0.001 [0.009]
ACF, current c as instrument	0.002 [0.010]	-0.007 [0.012]	0.024 [0.026]	0.002 [0.010]
ACF, lagged instruments only	0.003 [0.010]	0.002 [0.009]	0.002 [0.009]	0.003 [0.010]
IV (compute price) + path [‡]	0.005 [0.035]	-0.003 [0.032]	0.001 [0.032]	0.003 [0.032]
FOC system, T ignored	0.003 [0.009]	0.002 [0.010]	0.003 [0.009]	0.004 [0.010]
FOC system, T proxied	0.001 [0.009]	0.001 [0.009]	0.001 [0.009]	0.002 [0.009]
<i>Panel D. $\ln M^*(10^{24})$ (truth 2.898)</i>				
Pooled Huber, E free (ML practice)	0.53 [1.61]	0.55 [1.60]	-0.04 [1.41]	0.64 [1.50]
Pooled NLS + generation FE	0.01 [1.17]	0.23 [0.73]	0.10 [2.17]	0.27 [0.80]
Lab FE	0.00 [0.87]	0.11 [0.65]	-0.06 [1.35]	0.06 [0.83]
Lab $\times$ generation FE	0.00 [0.37]	-0.01 [0.38]	-0.01 [0.33]	-0.02 [0.39]
ACF, current c as instrument	0.03 [1.47]	0.10 [0.79]	0.52 [2.83]	-0.03 [1.50]
ACF, lagged instruments only	0.08 [1.58]	0.02 [1.51]	0.03 [1.50]	0.03 [1.60]
IV (compute price) + path [‡]	1.27 [1.27]	1.28 [1.28]	1.32 [1.32]	1.28 [1.28]
FOC system, T ignored	1.25 [1.25]	1.25 [1.25]	1.28 [1.28]	1.25 [1.25]
FOC system, T proxied	-0.02 [0.05]	-0.02 [0.05]	-0.03 [0.04]	-0.03 [0.05]

Note: Bias [root-mean-squared error] over 400 replications per compute rule. Simulated industry and compute rules: (a) exogenous budgets; (b) funding, $\ln C$ rises by $2x_{ft}$; (c) a loss target; (d) predetermined budgets, $\ln C$ rises by $2x_{f,t-1}$; setup in the text. E is known to every estimator except the first. Both ACF rows use current within-lab-generation deviations of n and d as instruments, valid here by construction; without them the lagged moment set is under-identified. [‡]Median bias [median absolute error], because just-identified 2SLS has no finite moments; median first-stage F is 17 to 18 under rules (a), (b), and (d) and 13 under (c). FOC system: the within-family loss equation stacked with the training-only first-order condition ($w = 1$), ignoring or proxying inference demand T (proxy with log noise of standard deviation 0.3). TFP growth: mean change in generation means of $y - \hat{F}(n, d)$. Every estimator is started from data-based values, none at the truth: nine random values for the pooled Huber fit, and nine combinations of $\alpha, \beta \in \{0.2, 0.5, 1\}$ with levels matched to mean output for the nonlinear least-squares estimators, keeping the lowest criterion; ACF starts from pooled NLS and the FOC systems from the lab $\times$ generation fit. *Source:* Author's simulations; replication code `code/analysis/m6_montecarlo`.

TABLE C2. TRANSMISSION BIAS IN THE RETURNS TO COMPUTE: CLOSED FORM AND SIMULATION

Compute rule		On the path ($T = 0$), $n = 10^6$			$T > 0$	$n = 240$	
		Formula	Simulated	Reverse	Simulated	Mean	Coverage (%)
Exogenous		0.1783	0.1781	0.542	0.1784	0.179	95
Funding, $\lambda =$	0.5	0.2091	0.2095	0.512	0.2099	0.209	53
Funding, $\lambda =$	1	0.2371	0.2370	0.481	0.2370	0.237	4
Funding, $\lambda =$	2	0.2783	0.2782	0.429	0.2784	0.278	0
Funding, $\lambda =$	4	0.3033	0.3034	0.359	0.3032	0.303	0
Predetermined, $\lambda =$	1	0.2253	0.2254	0.487	0.2257	0.225	16
Predetermined, $\lambda =$	2	0.2583	0.2582	0.435	0.2586	0.258	0
Predetermined, $\lambda =$	4	0.2783	0.2783	0.359	0.2789	0.278	0
Target, $\text{sd}(\bar{y}) =$	0	0.0000	0.0000	—	0.0000	0.000	0
Target, $\text{sd}(\bar{y}) =$	0.05	0.0069	0.0068	0.358	0.0068	0.007	0
Target, $\text{sd}(\bar{y}) =$	0.1	0.0246	0.0246	0.223	0.0246	0.025	0
Target, $\text{sd}(\bar{y}) =$	0.25	0.0891	0.0892	0.186	0.0886	0.089	0

Note: Slope of $y = -\ln(L - E)$ on $c = \ln C$ across labs. Formula: $\gamma + \pi_1 \text{Var}(\omega) / (\pi_1^2 \text{Var}(\omega) + \text{Var}(\eta))$ (Proposition 1(i)) with $\gamma = 0.1783$, $\text{sd}(\omega) = 0.25$, $\text{sd}(\eta) = 1$, persistence 0.8. Funding: $c = \lambda\omega + \eta$; predetermined: $c = \lambda\omega_{-1} + \eta$; target: labs buy the compute that reaches a target $\bar{y}$ with the stated dispersion (a common target gives slope 0). Simulated: OLS on 10^6 draws, seed noise 0.05 (simulation standard errors at most 0.0003). Reverse: $\text{Var}(y) / \text{Cov}(c, y)$; “—” where $\text{Cov}(c, y) = 0$ and it is undefined. $T > 0$: lifetime-optimal allocations with log-normal inference demand; under target rules an extra term of -0.002 to -0.001 appears because high- T labs buy more compute. $n = 240$: mean OLS slope and coverage of the nominal 95 percent OLS interval over 2,000 samples of 240 lab-generations; coverage rates are specific to $\text{sd}(\eta) = 1$ and rise with more exogenous dispersion in compute.

Source: Author’s simulations; replication code `code/analysis/m6_montecarlo`.

Combined with a funding rule ($\lambda = 2$), the net bias moves from $+0.100$ without selection to -0.006 and -0.046 with 50 and 75 percent dropped, so its sign is ambiguous, as the theory implies. The allocation identities with factor-augmenting productivity hold in the simulation to within 10^{-14} .

Reproducibility. The code is in the directory `code/analysis/m6_montecarlo` of the replication package. Its entry point is `run_v2.py`, and `designB_startcheck2.py` checks the starting values. Every design cell seeds its own random-number generator, so results do not depend on the number of processes.

TABLE D1. FEW-CLUSTER INFERENCE FOR THE FAMILY FIXED-EFFECTS BIASES

	θ_N bias = 0.203			θ_D bias = -0.262		
	SE Obs.	SE bias	p	SE Obs.	SE bias	p
CR1, all k (earlier version), normal	0.082	0.086	0.019	0.161	0.163	0.11
CR1, singletons dropped, normal	0.071	0.077	0.008	0.140	0.142	0.065
CR1, nested k , $t(18)$	0.055	0.063	0.005	0.110	0.112	0.031
CV3 jackknife, $t(18)$	0.052	0.059	0.003	0.104	0.106	0.024
WCR, benchmark fixed (earlier version)			0.046			0.024
WCR, benchmark fixed, nested k			0.044			0.025
WCR, with benchmark draws			0.048			0.028
WCU, with benchmark draws			<0.0001			0.010
WCR, family weights			0.031			0.024
WCR, model weights			0.010			0.030
WCR, CV3-studentized, with benchmark draws			0.048			0.011
Effective clusters G^* (of 19)		4.3			4.1	
Bias, one developer left out		[0.157, 0.224]			[-0.339, -0.193]	

Note: Family fixed-effects rows of Table 1, Panel B (HellaSwag, 57 models, benchmark including OLMo-2); only the inference differs across rows. ‘Earlier version’: the procedures of an earlier version of this analysis. SE (bias) adds the design-conditional bootstrap standard error of the benchmark (0.029 for θ_N , 0.020 for θ_D). The CR1 small-sample factor $\frac{G}{G-1} \frac{n-1}{n-k}$ is 2.24 on the variance when $k = 32$ counts the family dummies (15 of the 30 families are singletons), 1.64 with singletons dropped, and 1.04 when fixed effects nested within developer clusters are not counted (Cameron and Miller, 2015). A bootstrap- t is invariant to this constant; a normal reference is not, which is why the earlier version paired a p -value of 0.02 with a t -ratio of 1.6 for θ_D . WCR/WCU: restricted/unrestricted wild-cluster bootstrap- t , developer clusters, Webb weights, 9,999 draws (Cameron, Gelbach and Miller, 2008; Webb, 2023); subcluster: weights drawn by family or by model with developer-clustered standard errors (MacKinnon and Webb, 2018); CV3-studentized: the bootstrap statistic uses the CV3 standard error of Obs. – Exp. in the sample and in every draw. CV3: jackknife by developer (MacKinnon, Nielsen and Webb, 2023). G^* : Carter, Schnepel and Steigerwald (2017) with $\rho = 0$. Identifying variation for θ_D (partial leverage) by developer: Cerebras 37%, Microsoft 28%, Meta 12%, BigScience 7%; 7 of 19 developers contribute nothing (they have only singleton families) and 2 more under 0.1%; by family: Cerebras-GPT 37%, Phi 28%, OPT 9%. *Source:* Author’s calculations; see Table 1.

APPENDIX D. ADDITIONAL TABLES

Table D1 reports the few-cluster inference behind the family fixed-effects biases of Section III.E. Table D2 reports the comparisons of compute measures and the implied reliabilities of Section IV, and Table D3 the compute-only estimators: errors in variables, reverse regressions and instruments. Table D4 decomposes the small data exponent of the cross-lab model of Section VI, and Table D5 reports the compute-equivalent gains of transformers and the bias of frontier regressions.

TABLE D2. HOW WELL IS TRAINING COMPUTE MEASURED?

<i>Panel A. Discrepancy between two compute measures, $\ln(C_{alt}/6ND)$</i>					
	Models (fam.)	Mean	S.d.	95% interval	MAD s.d.
Hardware time (independent)	27 (12)	0.017	0.292	[0.197, 0.409]	0.296
Epoch ‘Hardware’/‘Reported’	14 (8)	0.074	0.195	[0.033, 0.244]	0.051
Epoch operation counting	51 (22)	0.000	0.129	[0.029, 0.205]	0.015
Developer-reported FLOP	5 (3)	−0.015	0.033	[0.000, 0.038]	0.025
<i>Panel B. Implied reliability of $\ln C$, $\lambda = 1 - \sigma_u^2 / \text{Var}(\ln C)$</i>					
	Models	σ_u	Pooled	Within family	Within, $n - 1$
Hardware-time s.d.	57	0.292	0.980	0.955	0.907
upper 95% bound	57	0.409	0.961	0.912	0.818
Epoch ‘Hardware’/‘Reported’	57	0.195	0.991	0.980	0.959
Epoch ‘Confident’ class	57	0.668	0.895	0.766	0.514
Hardware-time s.d.	20	0.292	0.973	0.969	0.947
upper 95% bound	20	0.409	0.948	0.939	0.895
Epoch ‘Hardware’/‘Reported’	20	0.195	0.988	0.986	0.976
Epoch ‘Confident’ class	20	0.668	0.861	0.838	0.721
Hardware-time s.d.	128	0.292	0.988	0.973	0.961
upper 95% bound	128	0.409	0.977	0.946	0.924
Epoch ‘Hardware’/‘Reported’	128	0.195	0.995	0.988	0.983
Epoch ‘Confident’ class	128	0.668	0.939	0.857	0.798

Note: Panel A compares $C = 6ND$ (reported N and D ; exact Hugging Face parameter counts where available) with other compute measures for main-sample models matched to Epoch (Epoch AI, 2026). Only hardware time is independent of reported N and D : chip-hours (or chips $\times$ training hours) $\times$ peak dense 16-bit FLOP/s $\times$ a common utilization of 0.3. Epoch’s utilization field is back-calculated from its own compute estimate for most rows and is not used. Epoch’s reported compute is itself $6ND$ (operation counting), a developer-reported count (usually $6ND$), or a geometric mean of $6ND$ and hardware time; the ‘Hardware’/‘Reported’ set, used as an independent comparison in an earlier version of this analysis, mixes all three. The hardware-time s.d. also contains the dispersion of true utilization and any error in chip-hours, so it bounds the error s.d. of $\ln 6ND$ from above; its within-family value is 0.208 (6 families with two or more records). Intervals: family-cluster bootstrap, 9,999 draws; MAD s.d.: $1.4826 \times$ median absolute deviation. Panel B: within-family variance $\sum \hat{e}_i^2 / (n - F)$, net of the F family means; the last column divides by $n - 1$, as an earlier version of this analysis did. Within-family reliabilities assume errors independent across a family’s models; an error in a family-wide token count is absorbed by the family effect.

Source: Author’s calculations.

TABLE D3. COMPUTE-ONLY ESTIMATORS: MEASUREMENT ERROR, REVERSE REGRESSION AND INSTRUMENTS

	Obs. (1)	Exp. (2)	Bias (3)	λ, R^2, F (4)	AR set; $\hat{\theta}/R^2$ (5)
<i>Panel A. HellaSwag, 57 models (benchmark including OLMo-2)</i>					
OLS, EIV, $\sigma_u = 0.29$	0.335 (0.038)	0.329 (0.007)	0.006 (0.039) [0.90]	0.980	
Family FE, EIV, $\sigma_u = 0.29$	0.461 (0.047)	0.409 (0.013)	0.052 (0.049) [0.30]	0.955	
Family FE, EIV, $\sigma_u = 0.67$	0.576 (0.059)	0.409 (0.013)	0.166 (0.060) [0.12]	0.766	
Family FE, EIV, $\sigma_u = 0.67, n - 1$	0.858 (0.088)	0.409 (0.013)	0.448 (0.089) [0.016]	0.514	
Reverse regression	0.560 (0.126)	0.352 (0.006)	0.209	0.585	0.328/0.585
Reverse, family FE	0.580 (0.122)	0.432 (0.012)	0.147	0.760	0.441/0.760
IV, frontier FLOP/\$ ($n = 57$)	0.724	0.012 (0.047)	0.712	0.90	$(-\infty, \infty)$
IV, frontier, developer FE ($n = 57$)	8.535	-2.287 (0.190)	10.822	0.01	$(-\infty, \infty)$
IV, own hardware ($n = 10$)	-0.090	0.195 (0.014)	-0.285	14.3	$[-1.50, 0.34]$ $[-0.94, 0.27]$
<i>Panel B. HellaSwag, 20 models (strictly experimental benchmark)</i>					
OLS, EIV, $\sigma_u = 0.29$	0.417 (0.019)	0.430 (0.007)	-0.013 (0.020) [0.44]	0.973	
Family FE, EIV, $\sigma_u = 0.29$	0.488 (0.055)	0.464 (0.010)	0.024 (0.056) [0.65]	0.969	
Reverse regression	0.551 (0.129)	0.442 (0.006)	0.109	0.737	0.406/0.737
Reverse, family FE	0.505 (0.081)	0.476 (0.009)	0.029	0.937	0.473/0.937
IV, frontier FLOP/\$ ($n = 20$)	0.008	0.384 (0.014)	-0.376	5.15	$(-\infty, \infty)$ $[-1.85, 0.27]$
IV, frontier, developer FE ($n = 20$)	-0.237	0.332 (0.017)	-0.569	3.09	$(-\infty, \infty)$ $(-\infty, \infty)$
IV, own hardware	fewer than 8 models with the instrument				

Note: Compute-only estimators of θ_C on the design-matched samples of Table 1. EIV: slope divided by the reliability $\lambda = 1 - \sigma_u^2 / \text{Var}(\ln C)$ (column 4), with $\text{Var}(\ln C)$ pooled or within family; the within-family variance is $\sum \hat{c}_i^2 / (n - F)$, net of the F family means. $\sigma_u = 0.29$ is the standard deviation of $\ln(C_{\text{hw}}/6ND)$ across the 27 main-sample models with hardware-time records (Table D2), an upper bound on the error in $\ln 6ND$; $\sigma_u = \ln 3/1.645$ is Epoch’s ‘Confident’ class, which describes the uncertainty of Epoch’s own estimate, not of $6ND$. ‘ $n - 1$ ’: the within-family variance computed with an $n - 1$ denominator, as in an earlier version of this analysis (reliability 0.514 instead of 0.766 for the same σ_u). Exp. applies the same estimator to the experimental outputs, which are built from the same measured inputs and need no correction. Brackets: restricted wild-cluster bootstrap- t p -values for zero bias (CR1-studentized, developer clusters, Webb weights, 9,999 draws, each with an independent draw of the benchmark). Reverse regression: $1/b$ from regressing $\ln C$ on the output. By construction $1/b = \hat{\theta}_C/R^2$ of the forward regression (column 5), so its gap to Exp. measures the output variance not explained by compute (productivity dispersion and noise), not a bias. IV: $\ln C$ instrumented by the log FLOP/s per dollar of the best data-center accelerator released six months before the model, or of the model’s own training hardware (Epoch); F : cluster-robust first-stage statistic; Anderson–Rubin sets by inversion over $\theta_C \in [-3, 4]$ with $t(G - 1)$ critical values (second line: normal critical values); $(-\infty, \infty)$: the set covers the whole grid.

Source: Author’s calculations; see Table 1; hardware records from Epoch AI (2026).

TABLE D4. WHY IS THE CROSS-LAB DATA EXPONENT SMALL? CHANNELS OF ATTENUATION

	n	$\hat{\alpha}$	$\hat{\beta}$	$\hat{\gamma}$	τ_C [95% CI]	Default stop	
						$\hat{\beta}$	τ_C
<i>Panel A. Cross-lab model on alternative data</i>							
Baseline (model 7)	231	0.138	0.025	0.021	6.1 [3.0, 26.5]	0.040	8.7
Benchmark-specific perplexity	231	0.139	0.023	0.020	5.7 [2.8, 21.2]	0.030	9.0
(a) Tokens seen, epochs = 1	231	0.065	0.035	0.023	8.0 [1.1, ∞]	0.018	2.6
(a) Tokens seen, epochs imputed	231	0.121	0.028	0.023	4.7 [2.4, ∞]	0.027	4.7
(a) Effective data	231	0.057	0.067	0.031	25.7 [3.9, ∞]	0.052	12.7
(a) Word-level vocabulary	134	0.040	0.000	0.000	n.d.	0.008	0.9
(a) Vocabulary control	185	0.135	0.021	0.018	3.2 [1.4, 18.7]	0.036	4.6
(b) Published 2018 or later	180	0.052	0.077	0.031	7.8 [2.0, ∞]	0.040	4.3
(b) Transformers only	104	0.061	0.039	0.024	5.2 [1.6, ∞]	0.032	11.3
(b) No year terms	231	0.063	0.084	0.036	–	0.037	–
(c) Benchmark-specific E	231	0.072	0.042	0.026	10.5 [2.7, 44.3]	0.042	10.5
(d) All models per paper	261	0.055	0.064	0.030	15.3 [2.7, 79.3]	0.038	7.6
(d) Top model per paper	144	0.143	0.025	0.022	6.7 [2.5, 56.9]	0.042	9.6
(d) Flagged outliers kept	238	0.150	0.023	0.020	5.2 [2.5, 17.9]	0.030	7.5
(d) No zero-shot evaluations	207	0.049	0.091	0.032	16.6 [3.1, 23.5]	0.059	11.4
<i>Panel B. Chinchilla sweep</i>							
E estimated, Huber	240	0.347	0.367	0.178	[0.169, 0.188]		
$E = 0$, Huber	240	0.375	0.061	0.053	[0.051, 0.055]		
$E = 0$, least squares	240	0.061	0.377	0.053			
Total-loss elasticities	240	0.053	0.056	0.054			
<i>Panel C. Monte Carlo on the cross-lab design</i>							
	Global optimum			MC		Authors' code	
	$\hat{\alpha}$	$\hat{\beta}$	$\hat{\gamma}$	τ_C [5, 95]		$\hat{\beta}$	τ_C
Correct specification	0.348	0.366	0.178	9.9 [7.4, 12.8]		0.366	12.0
(c) E dropped	0.046	0.388	0.041	11.2 [8.8, 15.5]		0.346	11.1
(c) and neutrality imposed	0.046	0.489	0.042	4.4 [3.7, 5.4]		0.446	4.2
(c)+(a) D = dataset size	0.032	0.389	0.029	10.4 [7.9, 16.5]		0.048	4.6
(c)+(a)+(d) best 3	0.036	0.415	0.033	10.9 [7.6, 18.1]		0.280	11.9
(c)+(a) random 3	0.033	0.398	0.030	10.7 [7.5, 17.9]		0.303	10.5
(c)+(a), $\rho(\psi_D, \ln D) = -0.5$	0.031	0.370	0.028	9.8 [5.3, 16.5]		0.045	4.6
(c)+(a), $\rho(\psi_D, \ln D) = +0.5$	0.025	0.384	0.023	10.5 [5.8, 21.4]		0.288	10.1

Note: Panel A: the model and penalized objective of [Ho et al. \(2024\)](#) (mean squared error plus $0.0025 \sum |\theta|$, $E = 0$), minimized to convergence on the indicated data; the last two columns (default stop) stop the optimizer at the default tolerance used in the authors' code. τ_C is the doubling time of effective compute, with a 95 percent paper-cluster bootstrap interval (400 draws, each started at zero; " ∞ " when nonpositive growth is inside the interval). Rows: (a) measurement of data, with D as tokens seen (missing epoch counts set to 1 or imputed from dataset size), as effective data under repetition ($R^* = 15.4$; [Muennighoff et al., 2023](#)), or with word-level vocabularies only or a vocabulary control; 33 percent of epoch counts are imputed; (b) time and collinearity; (c) benchmark-specific irreducible loss estimated by NLS; (d) selection, using all models of each paper, only the best, keeping flagged outliers, or dropping zero-shot evaluations. Panel B: the [Besiroglu et al. \(2024\)](#) estimator on the Chinchilla sweep with E estimated or set to zero, with 95 percent bootstrap intervals for $\gamma = \alpha\beta/(\alpha + \beta)$; the last row gives the sample means of the total-loss elasticities $\alpha u/L$, $\beta v/L$, and $\gamma(L - E)/L$ implied by the full model. Panel C: pseudo-true values (at the global optimum and with the authors' code) from noise-free data when the truth is the Besiroglu technology with Hicks-neutral progress ($\tau_C = 12$ months) evaluated at the actual inputs of the sample; MC median and 5th–95th percentiles of τ_C from 204 replications with noise of standard deviation 0.215; rows switch on (c) dropping E , (a) measuring D as dataset size, (d) keeping the best three models per paper, and a data-augmenting productivity shock ψ_D correlated with $\ln D$. n.d.: not defined because $\hat{\beta} = 0$.

Source: Data of [Ho et al. \(2024\)](#); Epoch AI digitization of [Hoffmann et al. \(2022\)](#); author's estimates.

TABLE D5. COMPUTE-EQUIVALENT GAINS OF TRANSFORMERS AND THE BIAS OF FRONTIER REGRESSIONS

<i>Panel A. Gain of transformers over other architectures</i>						
	Frontier elasticity		Difference	Gain $f(C)$ at $C =$		
	Other	Transformer	[95% CI]	10^{17}	10^{19}	10^{23}
Penalized	0.0133	0.0217	0.0085 [−0.012, 0.034]	0.02	0.45	163
Unpenalized	0.0197	0.0210	0.0013 [−0.011, 0.028]	10.6	14.5	27.1
<i>Panel B. Log loss on log compute, with and without a time trend</i>						
	Compute slope			Inflation	Compute	Implied
	Rows	No trend	Trend	[95% CI]	growth	s_A
All rows	231	0.0282 (0.0020)	0.0246 (0.0018)	1.15 [1.10, 1.21]	1.12	0.13
Record setters	36	0.0329 (0.0024)	0.0228 (0.0027)	1.44 [1.24, 1.78]	2.64	0.31

Note: Panel A: model 13 of [Ho et al. \(2024\)](#) (architecture-specific exponents, common factor-augmenting year terms, $E = 0$), estimated with the penalized objective run to convergence or by unpenalized NLS, on their 231-row sample, WikiText-103 in 2023. $f(C)$ is the compute a non-transformer lab needs to match the transformer frontier loss at compute C (FLOP), relative to C ; with $E = 0$, $\ln f$ is linear in $\ln C$ and constant only if the two frontier elasticities coincide, consistent with the condition in Corollary 1(i) (equal E and equal γ). Non-transformer models span $C \leq 3 \times 10^{18}$ FLOP, so f above that is extrapolated; bootstrap intervals of f span more than six orders of magnitude. Panel B: OLS of the log of log perplexity on $\ln(6ND)$ and benchmark indicators, with and without a linear publication-year trend; paper-clustered standard errors in parentheses; inflation is the ratio of the two slopes, with a paper-cluster bootstrap interval; compute growth is the within-sample growth of log compute per year; the implied algorithmic share s_A , one minus the ratio of the slope with trend to the slope without, is backed out of the inflation factor and is not an independent test. Record setters are releases with the best perplexity on the benchmark to date.

Source: Data of [Ho et al. \(2024\)](#); author's estimates.